



УНИВЕРСИТЕТ ЗА НАЦИОНАЛНО И СВЕТОВНО СТОПАНСТВО

Университет за национално и световно стопанство - София
Факултет: Приложна информатика и статистика
Специалност: Бизнес Информатика
2026 г.

Магистърска теза

на тема:

Облачни услуги - софтуер като услуга,
в рамките на определена организация

Изготвил:
Боян Антонов
bantonov_2520836@unwe.bg

Научен ръководител:
Доц. д-р Ваня Лазарова
vlazarova@unwe.bg

СЪДЪРЖАНИЕ

УВОД

ГЛАВА 1. ТЕОРЕТИЧНИ ОСНОВИ НА ОБЛАЧНИТЕ ТЕХНОЛОГИИ 7

1.1. Същност и еволюция на облачните изчисления

1.2. Основни характеристики на облачните среди (мащабируемост, еластичност, наличност)

1.3. Сервизни модели - инфраструктура като услуга, платформа като услуга, софтуер като услуга

1.3.1. Илюстративен модел за разграничаване на сервизните нива

1.4. Модели на разполагане и стратегии за доставка:

1.4.1. Публичен и частен облак

1.4.2. Хибриден облак

1.4.3. Многооблачен и поли-облачен подход: устойчивост и функционална гъвкавост

1.5. Предимства и недостатъци на облачните решения

1.6. Рискове и предизвикателства пред съвременните инфраструктури

1.7. Концептуална аналогия между облачната инфраструктура и градската екосистема

ГЛАВА 2. АРХИТЕКТУРНИ ПОДХОДИ ЗА УСТОЙЧИВИ СОФТУЕР КАТО УСЛУГА РЕШЕНИЯ 19

2.1. Концепции за многооблачни и поли-облачни архитектури

2.2. Сравнителен анализ: Монолитна среда срещу разпределена облачна среда

2.3. Проблемът „зависимост от доставчика“ и възможности за преодоляването му

2.4. Устойчивост и висока степен на наличност

2.5. Стратегии за възстановяване при аварии

2.6. Управление на инфраструктурата чрез код и ролята на Terraform

2.7. Преносимост на приложенията чрез контейнери и оркестрация

2.8. Съвременни тенденции и примери от практиката

ГЛАВА 3. БИЗНЕС АНАЛИЗ, РЕГУЛАЦИИ И ПРИЛОЖИМОСТ 35

3.1. Икономически аспекти и управление на разходите

3.2. Капиталови и оперативни разходи

3.3. Анализ на целевите групи организации:

3.3.1. Малки и средни предприятия - гъвкавост и оптимизация на ресурси

3.3.2. Големи предприятия - изисквания за сигурност и мащаб

3.4. Нормативни изисквания и защита на данните (GDPR, DORA)

3.5. Анализ на риска и ограничения на избрания подход

ГЛАВА 4. ПРОЕКТИРАНЕ НА РЕФЕРЕНТНА СОФТУЕР КАТО УСЛУГА АРХИТЕКТУРА ЗА
ПРИМЕРНА ОРГАНИЗАЦИЯ 44

4.1. Цели и обхват на архитектурата

4.2. Функционални и нефункционални изисквания

4.3. Обосновка на избора на технологии и доставчици

4.4. Общ архитектурен модел на решението

4.5. Компонентна структура и мрежова топология

4.6. Сценарии за отказ и механизми за устойчивост

4.7. Управление на достъпа и идентичността

ГЛАВА 5. РЕАЛИЗАЦИЯ НА ПРОТОТИПНА СИСТЕМА 55

5.1. Подход към реализацията и методологията

5.2. Конфигуриране на облачните среди и мрежова изолация

5.3. Разгръщане на изчислителни ресурси

5.4. Реализация на механизъм за автоматично пренасочване на трафика

5.5. Наблюдение, регистриране и известяване при проблеми

5.6. Технически ограничения на прототипа

ГЛАВА 6. ЕКСПЕРИМЕНТАЛНО ИЗСЛЕДВАНЕ И ОЦЕНКА 65

6.1. Методика на тестване

6.2. Работа при нормален режим

6.3. Симулация на отказ на доставчик

6.4. Измерване на времето за възстановяване

6.5. Сравнителен анализ на резултатите

6.6. Оценка на разходите за поддръжка

ЗАКЛЮЧЕНИЕ

ЛИТЕРАТУРА

УВОД

1. Актуалност на проблема

Днес облачните технологии се утвърдиха не просто като алтернатива на традиционната ИТ инфраструктура, а като фундаментален двигател на дигиталната трансформация. За съвременните организации, независимо от техния мащаб или индустрия, способността да предоставят и консумират дигитални услуги с висока скорост и надеждност е ключов фактор за конкурентоспособност. В този контекст моделът „Софтуер като услуга“ (Software as a Service - SaaS) се наложи като доминиращ стандарт за доставка на приложения. Според редица пазарни анализи [1], значителна част от корпоративния софтуер днес - от системите за управление на взаимоотношенията с клиенти (CRM) до платформите за планиране на ресурсите (ERP) - се използва под формата на Софтуер като услуга. Актуалността на настоящото изследване произтича от факта, че организациите стават критично зависими от тези услуги. Спирането на достъпа до ключова Софтуер като услуга платформа дори за няколко часа може да доведе до спиране на оперативната дейност, значителни финансови загуби и уронване на репутацията. Въпреки това, практиката показва, че разчитането на един единствен облачен доставчик крие сериозни рискове. Събития от последните години, при които водещи глобални доставчици претърпяха мащабни срывове, доказват, че „Облачните среди, въпреки високото ниво на резервираност, не са лишени от системни рискове.“ Това налага преосмисляне на архитектурните стратегии. Организациите вече не търсят просто „преминаване в облака“, а изграждане на устойчиви и гъвкави архитектури, които могат да гарантират непрекъсваемост на бизнеса. Тук на преден план излизат концепциите за многооблачни и поли-облачни подходи, които позволяват разпределяне на риска и използване на най-добрите функционалности от различни доставчици. Темата е особено значима и в контекста на новите регулаторни изисквания на Европейския съюз, които задължават финансовите и критичните институции да демонстрират високо ниво на цифрова устойчивост и стратегии за изход от зависимост.

2. Дефиниране на проблема

Основният проблем, който разглежда настоящата магистърска теза, е уязвимостта на монолитните Софтуер като услуга архитектури в рамките на организацията и свързаните с тях бизнес рискове. Когато една организация изгради своята Софтуер като услуга изцяло върху инфраструктурата на един доставчик (например само AWS или само GCP), тя се изправя пред следните предизвикателства:

- Единична точка на отказ: Дори при наличие на резервираност в различни зони, срыв на ниво регион или глобална грешка в контролния панел на доставчика може да доведе до пълно прекъсване на услугата.
- Зависимост от доставчика: Организацията става заложник на ценовата политика и техническата пътна карта на конкретния доставчик. Преместването на услугата към друг доставчик впоследствие става скъпо и технически сложно. [2]
- Липса на функционална гъвкавост: Ограничаване до инструментите, предлагани от един доставчик, което може да попречи на внедряването на иновации (напр. използване на специфичен ИИ модул от конкурентен облак).

Следователно, научният и практически проблем се състои в необходимостта от намиране на архитектурен баланс - как да се проектира Софтуер като услуга в организацията, така че тя да бъде максимално устойчива и независима, без това да води до прекомерно усложняване на управлението и неоправдано високи разходи.

3. Цел на дипломната работа

Целта на настоящата магистърска теза е да се анализират, проектират и експериментално проверят архитектурни подходи за изграждане на устойчива и гъвкава Софтуер като услуга в рамките на конкретна организация, използвайки възможностите на съвременните Много-облачни и Поли-облачни технологии.

Изследването цели да докаже, че чрез интелигентно разпределяне на инфраструктурата и услугите между два независими облачни доставчика може да се постигне по-висока степен на наличност и да се минимизират рисковете от прекъсване на дейността, в съответствие с най-добрите практики за управление на риска.

4. Задачи на изследването

За постигането на дефинираната цел са формулирани следните конкретни задачи:

- Да се направи задълбочен теоретичен анализ на концепциите за облачни услуги, моделите на доставка (SaaS) и видовете архитектури.
- Да се изследват бизнес аспектите и регулаторната рамка, включително икономическите модели (CAPEX/OPEX) и изискванията за дигитален суверенитет и защита на данните.
- Да се реализира прототипна система, разположена едновременно в средите на Amazon Web Services (AWS) и Google Cloud Platform (GCP), която демонстрира функционална свързаност и автоматизирано управление.
- Да се проведе експериментално изследване чрез симулация на критичен отказ (напр. спиране на единия облачен доставчик) и да се измерят показателите за възстановяване на услугата.

5. Обект и предмет на разработката

- **Обект на изследването** е информационната инфраструктура на съвременна организация, която предоставя или използва критични софтуерни услуги (SaaS).
- **Предмет на изследването** са архитектурните методи, технологии и инструменти (като Мулти-облачна оркестрация, Инфраструктура като код, Маршрутизиране на трафик чрез системата за домейн имена (DNS) за повишаване на устойчивостта, гъвкавостта и независимостта на тези услуги.

6. Методология на изследването

За изпълнение на поставените задачи е използвана комплексна методология, включваща:

- **Анализ на литературни източници:** Проучване на академични публикации, индустриални доклади и техническа документация.
- **Архитектурно моделиране:** Създаване на концептуални и логически модели на разпределена облачна среда.
- **Емпиричен подход:** Изграждане на работещ прототип и провеждане на контролирани експерименти за валидиране на теоретичните хипотези.

- **Сравнителен анализ:** Оценка на ефективността и разходите при различни архитектурни сценарии.

7. Структура на дипломната работа

Разработката е структурирана в шест глави. Първа глава разглежда теоретичните основи на облачните услуги и Софтуер като услуга модела. Втора глава анализира архитектурните подходи за постигане на устойчивост чрез Мулти-облачен и Поли-облачен. Трета глава е посветена на бизнес анализа, регулациите и икономическата приложимост в организациите. Четвърта глава представя проектирането на референтната архитектура, а пета описва практическата реализация на прототипа. Шеста глава съдържа резултатите от проведените експериментални изследвания и оценка на ефективността.

ГЛАВА 1. ТЕОРЕТИЧНИ ОСНОВИ НА ОБЛАЧНИТЕ ТЕХНОЛОГИИ

1.1. Същност и еволюция на облачните изчисления

Облачните изчисления (cloud computing) представляват съвременна парадигма за предоставяне на изчислителни ресурси като услуга чрез интернет, при която хардуерът, софтуерът и инфраструктурата се доставят на потребителите при поискване и с гъвкаво ценообразуване. В своята същност облачният модел променя традиционния начин на изграждане и управление на информационните системи, като измества фокуса от притежанието на физическа инфраструктура към използването на виртуализирани изчислителни ресурси. Исторически погледнато, концепцията за споделени изчислителни ресурси възниква още през 60-те години на XX век, когато се развиват първите модели за времево споделяне на изчислителни ресурси (time-sharing) в големите изчислителни центрове. Тогава идеята за централизирани изчислителни мощности, достъпни от множество потребители, поставя основата на по-късните облачни модели. През 90-те години с развитието на интернет и виртуализацията започва нов етап, при който се създават условия за динамично разпределяне на ресурси. Най-широко приетата дефиниция за облака е представена от Националния институт по стандарти и технологии на САЩ (NIST), според която моделът позволява удобен и повсеместен мрежов достъп до споделен пул от ресурси с минимални усилия за управление. [3]

Същинската трансформация настъпва в началото на XXI век, когато големи технологични компании започват да предлагат изчислителна инфраструктура като услуга чрез публични платформи. Този модел позволява на организациите да наемат изчислителна мощност, памет и мрежови ресурси без необходимост от изграждане на собствен център за данни.

Еволюцията на облачните изчисления може да бъде разгледана в няколко основни фази:

1. Централизирани изчислителни центрове
2. Виртуализация и споделяне на ресурси
3. Публични облачни платформи
4. Многооблачни и разпределени архитектури
5. Интелигентни облачни екосистеми с интегрирани AI услуги

С течение на времето облачният модел се утвърждава не само като технологично решение, а като стратегически инструмент за дигитална трансформация. Организациите преминават от капиталово-интензивен модел (CAPEX), базиран на закупуване на хардуер, към оперативен модел (OPEX), при който ресурсите се използват според текущите нужди.

1.2. Основни характеристики на облачните среди (мащабируемост, еластичност, наличност)

Облачните среди се отличават с редица специфични характеристики, които ги разграничават от традиционните IT инфраструктури. Сред тях ключово значение имат мащабируемостта, еластичността и наличността. [4]

Мащабируемост

Мащабируемостта представлява способността на една система да увеличава или намалява своите ресурси в зависимост от натоварването. В традиционна среда това изисква закупуване на допълнителен хардуер, инсталация и конфигурация, което е свързано със значителни разходи и време. В облачна среда този процес е автоматизиран. Организацията може динамично да добавя изчислителна мощност, памет чрез конфигурационни настройки или автоматични механизми за автоматично мащабиране (autoscaling). Разграничават се два типа мащабируемост:

- Вертикална (увеличаване на ресурсите на една машина)

- Хоризонтална (добавяне на нови инстанции „машини“)

Еластичност

Еластичността е способността на системата автоматично да се адаптира към промените в натоварването в реално време. Докато мащабируемостта описва потенциала за разширяване, еластичността описва динамичната реакция. Например при внезапен скок на потребителски трафик (напр. маркетингова кампания) системата може автоматично да увеличи броя на сървърните инстанции и след спад на трафика да ги намали, оптимизирайки разходите.

Наличност

Наличността е степента, до която дадена услуга остава достъпна за потребителите. Тя обикновено се измерва чрез проценти на коефициент на наличност (uptime) (например 99.9%, 99.99% и т.н.). За постигане на висока наличност облачните доставчици използват:

- Географско разпределение на ресурсите
- Зони за наличност (Availability Zones)
- Автоматично прехвърляне при отказ (failover)
- Репликация на данни

1.3. Сервизни модели - инфраструктура като услуга, платформа като услуга, софтуер като услуга

Облачните изчисления се характеризират със специфични модели за предоставяне на услуги, които дефинират степента на контрол, отговорност и управление между доставчика и крайния потребител. Тези модели структурират начина, по който се предоставят изчислителни ресурси, и създават ясна рамка за разпределение на отговорностите в облачната среда.

Трите основни сервизни модела са:

- Инфраструктура като услуга (Infrastructure as a Service - IaaS)
- Платформа като услуга (Platform as a Service - PaaS)
- Софтуер като услуга (Software as a Service - SaaS)

Инфраструктура като услуга (IaaS)

Инфраструктурата като услуга представлява най-ниското ниво в облачния сервизен модел. При него доставчикът предоставя виртуализирана инфраструктура - изчислителна мощност, памет и мрежови ресурси - докато клиентът управлява операционната система, приложенията и данните.

Този модел осигурява висока степен на гъвкавост и контрол, защото организациите могат да конфигурират средата според собствените си изисквания. Потребителят има възможност да създава виртуални машини, да определя мрежови политики и да инсталира специфичен софтуер.

IaaS е подходящ при:

- миграция на съществуващи системи
- разработка и тестова среда
- изграждане на мащабируеми уеб приложения
- внедряване на корпоративни решения

Предимството на този модел е намаляването на капиталовите разходи, защото не е необходимо изграждане на собствен център за данни. Недостатък може да бъде по-високата отговорност за сигурността и конфигурацията на системата.

Платформа като услуга (PaaS)

Платформата като услуга представлява междинен модел, при който доставчикът предоставя не само инфраструктура, но и среда за разработка, изпълнение и управление на приложения.

При този модел доставчикът управлява:

- хардуера
- виртуализационния слой
- операционната система
- middleware компонентите

Потребителят се фокусира върху разработката на приложенията и управлението на данните.

РaaS улеснява разработчиците, като премахва необходимостта от конфигуриране и поддръжка на базовата инфраструктура. Това ускорява процеса на разработка и внедряване (deployment), като същевременно намалява оперативната сложност.

Основните предимства включват:

- ускорен жизнен цикъл на разработка
- стандартизирана среда
- по-нисък риск от инфраструктурни грешки

Ограничението при този модел е зависимостта от конкретния доставчик и по-малката степен на контрол върху системната конфигурация.

Софтуер като услуга (SaaS)

Софтуерът като услуга представлява най-високото ниво на абстракция в облачната архитектура. При този модел потребителят получава достъп до готово приложение чрез интернет, без да се ангажира с инсталация, поддръжка или инфраструктурно управление.

Всички технически аспекти се управляват от доставчика:

- инфраструктура
- платформа
- приложение
- сигурност и актуализации

SaaS решенията се характеризират с:

- достъп чрез браузър
- абонаментен модел на плащане
- централизирана поддръжка
- автоматични обновления

Този модел е особено подходящ за бизнес приложения, системи за управление на взаимоотношения с клиенти (CRM), системи за управление на ресурси (ERP) и други платформи.

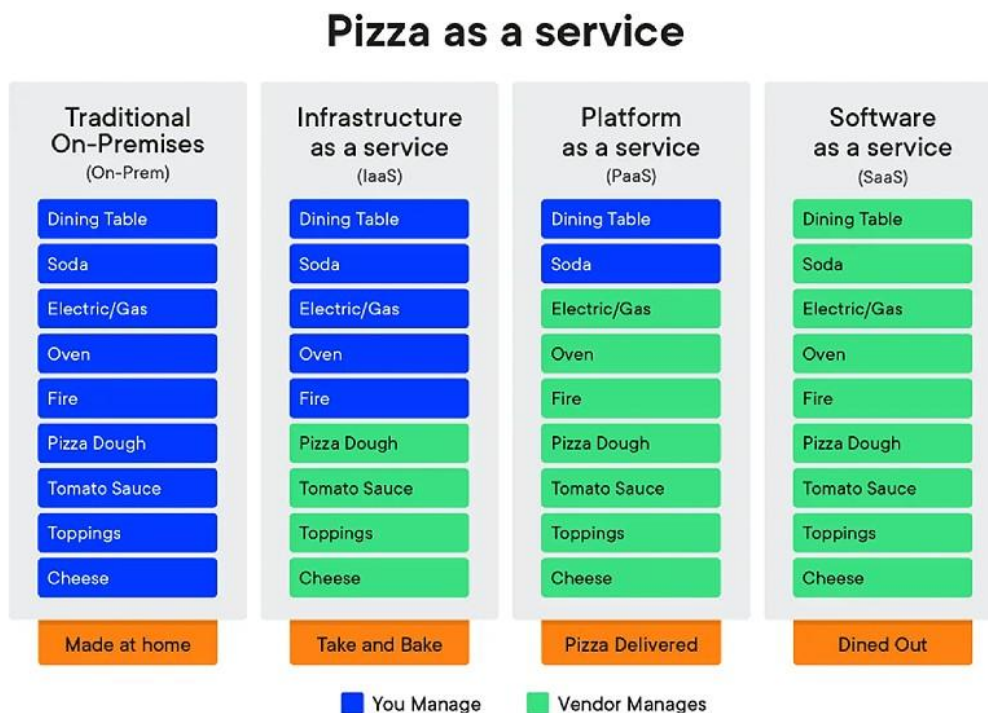
Сравнителен анализ на сервизните модели:

Основната разлика между трите модела се състои в степента на контрол и отговорност.

Ниво	Контрол от страна на клиента	Управление от доставчика
IaaS	Висок	Базова инфраструктура
PaaS	Среден	Инфраструктура + платформа
SaaS	Нисък	Цялостна система

С нарастване на нивото на абстракция намалява техническата сложност за клиента, но се увеличава цената за заплащане и зависимостта от доставчика.

1.3.1. Илюстративен модел за разграничаване на сервизните нива



В практиката често се използва популярна илюстративна аналогия, известна като „Pizza as a Service“, за да се демонстрира разликата между различните нива на отговорност в облачните сервизни модели.

В тази аналогия:

- При традиционния модел (on-premise) потребителят закупува всички съставки, разполага със собствена кухня, оборудване и носи пълна отговорност за целия процес - от приготвянето до почистването. (Направена вкъщи)
- При инфраструктура като услуга (IaaS) доставчикът предоставя „кухнята“ и оборудването, но клиентът сам приготвя пицата. (Взимаш замразена)
- При платформа като услуга (PaaS) доставчикът предоставя готова кухня и полуготови съставки, а клиентът се фокусира основно върху „рецептата“. (Поръчваш си пица)
- При софтуер като услуга (SaaS) потребителят получава готова пица, без да участва в процеса на приготвяне. (Излизаш на пицария)

Тази аналогия ясно демонстрира как с увеличаване на нивото на абстракция намалява степента на техническа отговорност на клиента, но паралелно с това се увеличава зависимостта от доставчика.

1.4. Модели на разполагане и стратегии за доставка

Освен сервизните модели, облачните технологии се класифицират и според начина на разполагане (deployment model). Този аспект определя къде се намира инфраструктурата и какъв е достъпът до нея.

1.4.1. Публичен и частен облак

Публичният облак представлява среда, в която ресурсите се предоставят на множество организации чрез споделена инфраструктура. Тя се управлява от външен доставчик и е достъпна чрез интернет.

- Основни характеристики:
 1. споделена инфраструктура
 2. гъвкаво ценообразуване
 3. висока мащабируемост
 4. минимални първоначални инвестиции
- Подходящ е за:

1. стартиращи компании
2. динамични бизнес среди
3. системи с променливо натоварване

Частният облак представлява инфраструктура, предназначена за използване от една организация. Той може да бъде разположен в собствен център за данни или управляван от външен доставчик, но остава изцяло изолиран.

- Предимства:
 1. по-високо ниво на контрол
 2. персонализирани политики за сигурност
 3. съответствие с регулаторни изисквания
 4. Недостатък са по-високите разходи и необходимостта от вътрешна експертиза.

1.4.2. Хибриден облак

Хибридният модел комбинира публична и частна облачна инфраструктура в единна архитектура. Той позволява на организациите да разпределят натоварването според чувствителността на данните и бизнес изискванията.

- Например:
 1. чувствителни данни се съхраняват в частен облак
 2. публични уеб услуги се разполагат в публичен облак
 3. този модел осигурява баланс между сигурност и мащабируемост.

1.4.3. Многооблачен и поли-облачен подход: устойчивост и функционална гъвкавост

- Многооблачният (multi-cloud) подход включва използване на услуги от повече от един публичен облачен доставчик. Целта е да се избегне зависимостта от един доставчик и да се повиши устойчивостта.
- Поли-облачният (poly-cloud) модел отива една стъпка по-далеч, като използва специфичните предимства на различни доставчици за различни функционални цели - например един доставчик за аналитични услуги и друг за машинно

обучение.

- Предимства:
 1. повишена устойчивост при отказ
 2. оптимизация на разходите
 3. функционална специализация
- Предизвикателства:
 1. повишена архитектурна сложност
 2. необходимост от унифицирано управление
 3. интеграционни трудности

1.5 Предимства и недостатъци на облачните решения

Внедряването на облачни услуги променя фундаментално икономическия и оперативен модел на съвременната организация. Предимствата и недостатъците не са просто технически параметри, а стратегически фактори, влияещи върху конкурентоспособността.

Предимства на облачния модел:

- Икономическа ефективност и оптимизация на разходите: Основен стимул е преходът от модел на големи капиталови инвестиции (CAPEX) към модел на текущи оперативни разходи (OPEX). Моделът „плащаш за това, което ползваш“ (pay-as-you-go) елиминира нуждата от преоразмеряване (over-provisioning) на хардуера. Това води до намаляване на общата цена на притежание (TCO - Total Cost of Ownership) на ИТ инфраструктурата.
- Гъвкавост и бързина на иновациите (Agility): Облакът предоставя достъп до готови екосистеми за анализ на големи данни (Big Data), автоматизация на жизнения цикъл на софтуера (DevOps), контейнеризация и изкуствен интелект (AI). Това позволява на организациите да съкратят драстично времето за пускане на нови продукти на пазара (Time-to-Market).
- Висока наличност и непрекъсваемост на бизнеса: Географското разпределение на центровете за данни и вградените механизми за резервираност позволяват постигане на нива на наличност (uptime), които са непосилни за повечето частни центрове за данни. Автоматизираното възстановяване при отказ (failover) гарантира, че критичните SaaS услуги остават достъпни дори при локални технически сринове.

- Фокус върху основната дейност: При моделите PaaS и SaaS техническата поддръжка на инфраструктурата, сигурността на физическия слой и актуализациите са отговорност на доставчика. Това освобождава човешки ресурс в организацията за иновации и оптимизация на бизнес процесите.

Недостатъци и ограничения:

- Пълна зависимост от интернет свързаността: Облачните услуги са свързани с качеството на мрежовата връзка. Прекъсвания или висока латентност (latency) могат да направят бизнес процесите неизпълними, което изисква инвестиции в резервни интернет канали.
- Загуба на директен контрол: Използването на публичен облак означава, че организацията няма контрол върху физическия хардуер и специфичните настройки на хипервайзера. Това може да бъде пречка при специфични софтуерни изисквания или при одити за съответствие.
- Сложност на управлението на разходите: Макар и гъвкав, облачният модел може да доведе до „разрастване на ресурсите“ (Cloud Sprawl) и неочаквано високи месечни сметки, ако липсват строги политики за мониторинг.

1.6. Рискове и предизвикателства пред съвременните инфраструктури

В съвременната динамична среда управлението на облачната инфраструктура е свързано с комплексни рискове, които изискват нов подход към киберсигурността и архитектурното проектиране.

Технологични и стратегически рискове

- Технологична зависимост (Vendor Lock-in): Това е един от най-сериозните рискове за организацията. Използването на API интерфейси, специфични формати на данни и уникални облачни услуги прави миграцията към друг доставчик изключително скъпа и технически трудна. Този риск е и основният двигател за преминаване към Multi-cloud стратегии.
- Архитектурна сложност и интеграция: Комбинирането на различни облачни доставчици и свързването им с локални системи увеличава сложността на мрежовата топология. Осигуряването на сигурна и бърза комуникация между AWS и GCP например изисква

експертни познания и правилно конфигуриране на VPN тунели или директни връзки.

Сигурност и споделена отговорност

- **Моделът на споделената отговорност (Shared Responsibility Model):** Често организациите погрешно вярват, че облачният доставчик е изцяло отговорен за сигурността. Реалността е, че доставчикът отговаря за сигурността на облака, но клиентът отговаря за сигурността в облака (данни, конфигурации, идентичност).
- **Грешни конфигурации и човешки фактор:** Статистически, най-големият риск от изтичане на данни произтича от грешки в конфигурацията на правата за достъп (IAM) или неправилно конфигурирани публични „хранилища“ (S3 buckets).
- **Кибератаки и заплахи:** Облачните инфраструктури са постоянна мишена за DDoS атаки, фишинг и опити за неправомерен достъп. Осигуряването на криптиране на данните в покой (at rest) и при пренос (in transit) е императивно изискване.

Регулаторни и организационни предизвикателства

- **Дигитален суверенитет и съответствие (Compliance):** Регулации като GDPR и DORA налагат строги правила за това къде се съхраняват данните и как се гарантира тяхната устойчивост. Организациите трябва да са сигурни, че техният SaaS модел отговаря на местното законодателство.
- **Дефицит на умения и културна промяна:** Преходът към облачни услуги налага преквалификация на персонала в области като автоматизация (Terraform), контейнеризация (Kubernetes) и мониторинг. Често организационната съпротива срещу промяната е по-голямо предизвикателство от самото технологично внедряване.

1.7. Облачната инфраструктура като комплексна система -аналогия с градската екосистема

За да се осмисли напълно мащабът, архитектурната сложност и бизнес стойността на съвременните облачни технологии, е изключително полезно те да бъдат разгледани през призмата на една концептуална метафора. Най-точният еквивалент на облачната парадигма във физическия свят е функционирането на един съвременен град.

Тази аналогия премахва абстракцията на ИТ терминологията и позволява визуално и логически да се разбере защо преминаването към многооблачни (Multi-cloud) архитектури е критично за съвременните SaaS приложения.

Елементите на градската екосистема и техният облачен еквивалент:

- Хората / Гражданите (Крайните потребители - Users): Това са клиентите или служителите на организацията. В един град хората се разхождат по улиците с конкретна цел -искат да си купят хляб, да изпият кафе или да се подстрижат. Тях изобщо не ги интересува от коя електроцентрала идва токът в обекта, нито по какви тръби тече водата. В дигиталния свят, това е потребителят, който отваря браузъра си и очаква SaaS услугата просто да работи бързо и безпроблемно.
- Обектите (SaaS приложенията - Apps): Това са магазините, кафенетата, бръснарниците и офисите в града. Те генерират реалната бизнес стойност и обслужват хората. В нашата ИТ среда това са самите софтуерни приложения (системи за продажби, портали за документи, ERP системи). За да може обаче бръснарницата да отвори врати и да работи, тя е абсолютно зависима от външни ресурси.
- Инфраструктурата на града (IaaS / Cloud Infrastructure): Това са пътищата, уличното осветление, водопроводът и електрическата мрежа. Една бръснарница не може да функционира без ток за машинките и топла вода за измиване. Един магазин се нуждае от пътища, по които да му доставят стоката. По абсолютно същия начин, всяко SaaS приложение се нуждае от изчислителна мощност (ток/CPU), памет (складове/Storage) и мрежова свързаност (пътища/Networking), за да съществува. Тези базови "комунални услуги" се предоставят от облачния доставчик (AWS, GCP).

Защо тази аналогия доказва нуждата от Multi-cloud подход?

Когато една организация избере традиционния модел с един облачен доставчик (Single-cloud), тя на практика отваря своя "магазин" в град, който разчита на една единствена електроцентрала и една ВиК компания. Ако в града стане тежка авария и токът спре, магазинът затваря. Потребителите (users) идват, виждат табела "Затворено" и си тръгват недоволни. За бизнеса това

означава директна загуба на пари и репутация.

Точно тук се намесват архитектурите, обект на настоящата разработка:

1. Многооблачен модел (Multi-cloud): Това е еквивалентът на това да подсигурирм нашия обект стратегически. Ние свързваме магазина си към главната мрежа на Град А (напр. AWS), но изграждаме и резервна връзка към мрежата на съседния Град Б (напр. GCP). Ако в Град А спре токът, нашето табло автоматично превключва на захранване от Град Б. Светлините в магазина дори не примигват, хората вътре продължават да пазаруват и изобщо не разбират, че е имало авария. Това осигурява критичната за бизнеса устойчивост и непрекъсваемост (High Availability).
2. Поли-облачен модел (Poly-cloud): Това е еквивалентът на това да изберем "най-доброто от двата свята". Собственикът на обекта преценява, че ще ползва електричество от Град А, защото там имат най-модерните и евтини соларни паркове, но ще ползва водопровода на Град Б, защото тяхната вода е най-чиста. В дигиталния свят, това означава организацията да ползва сървърите на AWS, но алгоритмите за изкуствен интелект на Google, постигайки максимална функционална гъвкавост (Best-of-breed).

Разглеждайки облака като жива, взаимосвързана инфраструктурна екосистема, става ясно, че успешното предоставяне на надеждни SaaS услуги в една организация не е просто въпрос на писане на код, а на интелигентно архитектурно планиране. За да не "затваря магазинът" при всяка авария при доставчика, ИТ инфраструктурата се нуждае от резервни маршрути, оркестрация и автоматизирано управление.

ГЛАВА 2. АРХИТЕКТУРНИ ПОДХОДИ ЗА УСТОЙЧИВИ СОФТУЕР КАТО УСЛУГА РЕШЕНИЯ

След разглеждането на концептуалните основи, предимствата и рисковете на облачните инфраструктури в предходната глава, настоящата част се фокусира върху архитектурните модели и инженерните практики, които осигуряват устойчивост, преносимост и висока наличност на Software as a Service (SaaS) решения.

В контекста на нарастваща сложност, многооблачни среди и динамично потребление на ресурси, устойчивата архитектура се превръща в ключов фактор за дългосрочната жизнеспособност на дигиталните услуги.

2.1. Концепции за многооблачни и поли-облачни архитектури

Нарастващата сложност на съвременните софтуерни системи и стремежът към висока устойчивост, гъвкавост и технологична независимост водят до възприемането на архитектурни модели, базирани на използването на повече от един облачен доставчик. В този контекст се разграничават концепциите за многооблачна (multi-cloud) и поли-облачна (poly-cloud) архитектура, които представляват стратегически подход към изграждането на съвременни SaaS решения.

Многооблачната архитектура представлява използване на услуги от два или повече независими облачни доставчика в рамките на една организация. Обикновено това включва комбинация от водещи платформи като:

- Amazon Web Services
- Microsoft Azure
- Google Cloud Platform

В класическия multi-cloud модел различни компоненти на системата могат да бъдат разположени в различни облаци - например бекенд услуги в една среда, аналитични инструменти в друга, а резервни копия в трета.

Основни цели на многооблачния подход:

- намаляване на риска от пълен инфраструктурен отказ
- избягване на зависимост от един доставчик
- оптимизация на разходите чрез ценова конкуренция
- използване на специфични предимства на отделните платформи

Многооблачната стратегия често се възприема като инструмент за управление на риска, особено в организации с високи изисквания за непрекъсваемост на услугите.

Поли-облачна архитектура (Poly-cloud) - стратегическа интеграция на специализирани услуги. Докато multi-cloud се фокусира върху използването на повече от един доставчик, поли-облачната архитектура предполага съзнателен избор на най-добрите специализирани услуги от различни облаци в рамките на една интегрирана система.

Например:

- използване на една платформа за машинно обучение,
- друга за управление на бази данни,
- трета за съдържателна дистрибуция или edge услуги.

В този модел архитектурата се изгражда около принципа „best-of-breed“, при който се избира най-подходящата услуга за конкретната функционалност, независимо от доставчика.

Поли-облачният подход предполага по-висока степен на архитектурна зрялост и изисква:

- стандартизирани интерфейси
- API-базирана интеграция
- централизирано наблюдение и логване
- унифицирани политики за сигурност

Съществуват няколко основни модела за реализация на многооблачни архитектури:

1. Разделяне по функционалност

Различни бизнес функции се разполагат в различни облаци.

2. Активно-активна конфигурация

Идентични услуги се изпълняват паралелно в два облака, осигурявайки висока наличност и баланс на натоварването.

3. Активно-пасивна конфигурация

Основната среда работи в един облак, а вторичната служи за резервираност и възстановяване при аварии.

4. Федеративен модел

Независими среди комуникират чрез стандартизирани интерфейси и споделени протоколи.

Изборът на модел зависи от бизнес изискванията, регулаторните ограничения допустимия риск.

Предимства на многооблачните и поли-облачните архитектури:

1. Повишена устойчивост

Разпределянето на услугите между различни доставчици намалява вероятността от пълен отказ.

2. Технологична гъвкавост

Организацията не е ограничена от технологичния стек на един доставчик.

3. Оптимизация на разходите

Изборът на конкретна услуга според ценовия модел може да доведе до по-добра икономическа ефективност.

4. Подобро съответствие с регулации

Данните могат да бъдат разпределяни според географските и нормативните изисквания.

Въпреки стратегическите си предимства, многооблачните архитектури създават значителни инженерни предизвикателства:

- Повишена сложност
- Управлението на различни интерфейси, инструменти и политики увеличава оперативното натоварване.
- Липса на унифицирано управление
- Различните облаци предлагат собствени механизми за сигурност, мониторинг и таксуване.
- Проблеми със сигурността
- Синхронизирането на политики за достъп и криптиране между платформи изисква централизирана стратегия.
- Разходи за интеграция
- Международната свързаност и трансферът на данни между облаци могат да доведат до допълнителни разходи.

Внедряването на многооблачна стратегия не следва да бъде резултат от технологична мода, а от внимателен стратегически анализ. Необходимо е да бъдат оценени:

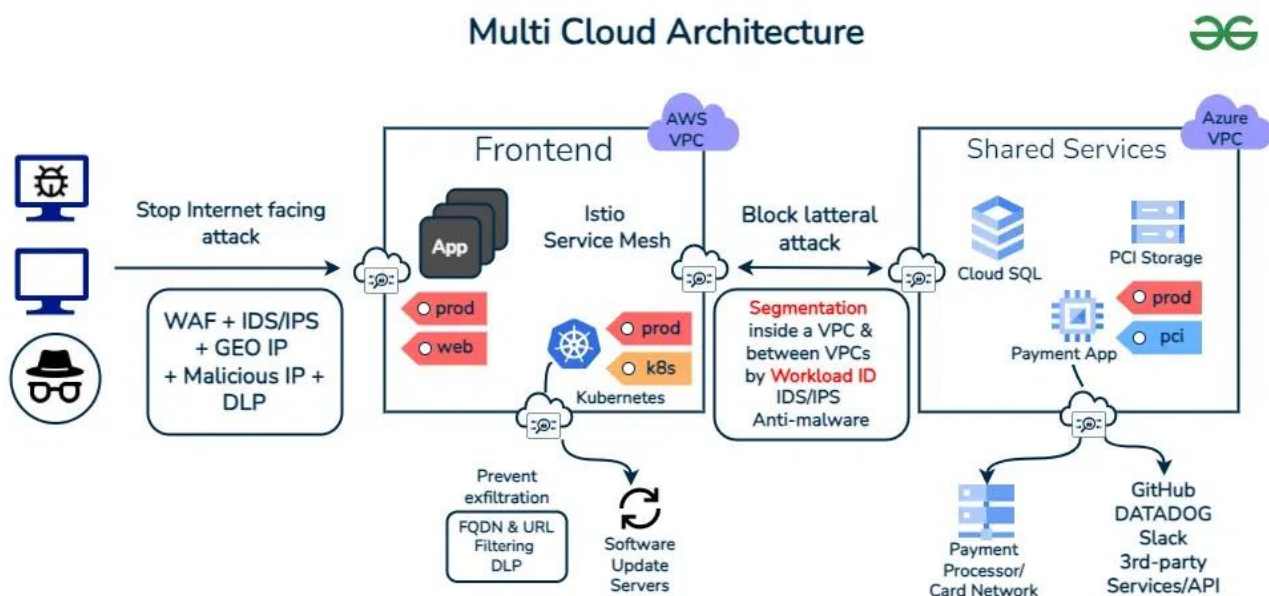
- критичността на услугите
- изискванията за наличност
- регулаторната рамка

- наличният експертен капацитет
- общата стойност на притежание (TCO)

Организациите, които успешно прилагат многооблачни модели, обикновено изграждат централизирани практики за управление на облачните ресурси (Cloud Governance) и автоматизират инфраструктурата чрез Infrastructure as Code.

Многооблачните и поли-облачните архитектури представляват логичен етап в еволюцията на облачните технологии. Те предлагат по-висока степен на устойчивост, гъвкавост и независимост, но изискват зряла архитектурна стратегия и значителна експертиза. В контекста на устойчивите SaaS решения, многооблачният подход не е само технологично решение, а стратегически инструмент за управление на риска, разходите и иновациите.

Примерна референтна архитектура на многооблачна SaaS среда:



- източник: [geeksforgeeks.org](https://www.geeksforgeeks.org)

Представената архитектура илюстрира реализация на многооблачен модел, при който фронтенд компонентите са разположени в отделна виртуална мрежа (VPC) от споделените услуги. Наблюдава се ясно разграничение между производствена (prod) и чувствителна (PCI) среда.

Архитектурата включва следните ключови елементи:

- Web Application Firewall (WAF) и IDS/IPS механизми за защита от външни атаки
- Kubernetes клъстер с service mesh за управление на вътрешната комуникация
- Сегментация на мрежата за ограничаване на странично придвижване (lateral movement)
- Интеграция с външни платежни мрежи и API услуги
- Централизиран мониторинг чрез външни системи

Този модел демонстрира как многооблачната стратегия може да бъде комбинирана с принципите на нула-вяра (zero-trust) сигурност и висока степен на наличност.

2.2. Сравнителен анализ: Монолитна среда срещу разпределена облачна среда

Еволюцията на софтуерните архитектури е пряко свързана с развитието на облачните технологии и необходимостта от изграждане на устойчиви, мащабируеми и гъвкави SaaS решения. В този контекст се открояват два основни архитектурни подхода - монолитният модел и разпределената облачна архитектура, често реализирана чрез микросървисен принцип.

Монолитната архитектура представлява централизирана система, при която всички функционални компоненти на приложението са интегрирани в единна кодова база и се изпълняват като един процес. Обикновено тя използва една централизирана база данни и се внедрява като единен артефакт. Този модел е характерен за традиционните информационни системи и за SaaS решения в начален етап на развитие. Основното предимство на монолитния подход е по-ниската първоначална сложност. Разработката, тестването и внедряването са относително по-прости, а комуникацията между отделните модули не изисква мрежова инфраструктура. В ранните фази на проект с ограничен обхват този модел може да бъде напълно достатъчен и икономически оправдан. С увеличаване на потребителската база и функционалността обаче монолитната архитектура започва да проявява ограничения.

Мащабирането обикновено е вертикално - чрез добавяне на ресурси към една машина - което има физически и финансови граници. Отказ в един компонент може да доведе до недостъпност на цялата система. Освен това внедряването на нови функционалности често изисква повторно деплойване на цялото приложение, което увеличава риска от грешки и прекъсвания.

За разлика от това, разпределената облачна архитектура разделя системата на независими услуги, които комуникират чрез API интерфейси. Този модел често се реализира чрез

контейнеризация и оркестрация с технологии като Docker и Kubernetes. Всяка услуга може да бъде разработвана, внедрявана и мащабирана независимо от останалите.

Основното предимство на разпределения модел е възможността за хоризонтално мащабиране на отделни компоненти в зависимост от натоварването. Това позволява по-ефективно използване на ресурсите и значително повишава устойчивостта на системата. При отказ на отделен микросървис останалите услуги могат да продължат да функционират, което минимизира риска от пълен срив. Разпределените архитектури също така осигуряват по-голяма технологична гъвкавост. Различните услуги могат да бъдат разработени с различни програмни езици и технологични стекове, което позволява избор на оптимален инструмент за всяка функционалност. Това подпомага иновацията и намалява дългосрочните технологични ограничения. Наред с предимствата обаче, разпределените среди въвеждат значителна архитектурна сложност. Управлението на комуникацията между услугите, мониторингът, логването, сигурността и управлението на конфигурации изискват висока степен на DevOps знания и специалисти. Наличието на множество независими компоненти увеличава броя на потенциалните точки на отказ и налага използването на централизирани инструменти за наблюдение и автоматизация.

В контекста на устойчивите SaaS решения изборът между монолитна и разпределена архитектура следва да бъде стратегически обоснован. Монолитният модел може да бъде подходящ при ограничен мащаб и ниска сложност, докато разпределените облачни архитектури предоставят значително по-добри възможности за мащабируемост, устойчивост и глобална достъпност. С нарастването на изискванията за непрекъсваема работа и бърза адаптация към пазарни промени, микросървисният подход постепенно се утвърждава като предпочитан архитектурен стандарт за съвременните SaaS платформи.

2.3. Проблемът „зависимост от доставчика“ и възможности за преодоляването му

Зависимостта от доставчика (vendor lock-in) представлява стратегически риск при изграждането на облачни SaaS решения. Тя възниква, когато дадена организация стане силно обвързана с конкретна облачна платформа до степен, при която миграцията към алтернативен доставчик е свързана с високи финансови разходи, технологични ограничения или оперативни рискове.

Водещи облачни платформи като Amazon Web Services, Microsoft Azure и Google Cloud Platform предоставят широк спектър от специализирани услуги, които значително ускоряват разработката на приложения. Въпреки това, използването на специфични технологии може да доведе до силна технологична обвързаност.

Зависимостта може да се прояви в няколко направления:

- използване на специфични за доставчика API интерфейси
- внедряване на специални бази данни или съобщителни услуги
- използване на специфични механизми за удостоверяване и управление на идентичности
- изграждане на архитектура, базирана на уникални инфраструктурни услуги
- договорни ограничения и специфични ценови модели

Последиците от подобна зависимост включват:

- високи разходи при миграция
- ограничена гъвкавост при стратегически промени
- повишен риск при промяна на ценовата политика
- трудности при интеграция с други платформи

В контекста на устойчивите SaaS решения управлението на този риск изисква целенасочена архитектурна стратегия. Сред основните подходи за минимизиране на vendor lock-in се открояват:

- използване на отворени стандарти и протоколи
- контейнеризация на приложенията чрез технологии като Docker
- оркестрация чрез платформи като Kubernetes, които могат да функционират в различни облачни среди
- управление на инфраструктурата чрез код с инструменти като Terraform
- прилагане на абстракция между приложната логика и облачните услуги
- внедряване на многооблачна стратегия

Особено ефективен подход е изграждането на cloud-agnostic архитектура, при която бизнес логиката е максимално независима от конкретната инфраструктура. Това позволява относително безпроблемна миграция между различни облачни среди и намалява стратегическия риск.

В заключение, зависимостта от доставчика не следва да бъде разглеждана като неизбежен резултат от използването на облачни технологии, а като управляем архитектурен риск. Чрез правилно проектиране, стандартизация и автоматизация организациите могат да съчетаят предимствата на специализираните облачни услуги с необходимата степен на технологична независимост.

2.4. Устойчивост и висока степен на наличност

Устойчивостта и високата степен на наличност са фундаментални изисквания към съвременните SaaS решения. В глобална дигитална среда потребителите очакват непрекъсваем достъп до услугите, независимо от географското им местоположение или моментното натоварване на системата.

Наличността на една система обикновено се измерва чрез проценти на достъпност (например 99.9%, 99.99% и т.н.), като всяко допълнително „девет“ значително намалява допустимото време на аварията. Постигането на висока степен на наличност изисква комбинация от архитектурни, инфраструктурни и организационни мерки.

Ключови принципи за изграждане на устойчива архитектура включват:

- резервираност на критичните компоненти
- елиминиране на единични точки на отказ (single point of failure)
- хоризонтално мащабиране
- автоматично възстановяване при отказ
- постоянен мониторинг и реакция в реално време

В облачна среда устойчивостта често се реализира чрез:

- разполагане на ресурси в множество availability зони
- географско разпределение на инфраструктурата
- използване на load balancer механизми

- автоматично преразпределение на натоварването
- репликация на данни в реално време

Контейнерните платформи като Kubernetes допринасят значително за устойчивостта чрез автоматично рестартиране на отказали контейнери, динамично мащабиране и поддържане на желано състояние на системата (desired state management).

Освен техническите механизми, устойчивостта изисква и ясно дефинирани показатели като:

- RTO (Recovery Time Objective) - максимално допустимо време за възстановяване
- RPO (Recovery Point Objective) - допустима загуба на данни във времеви интервал

Проектирането на система с висока наличност предполага баланс между разходи и риск.

Архитектура с активно-активна конфигурация в различни региони предлага най-висока степен на устойчивост, но е свързана със значително по-високи оперативни разходи. Следователно изборът на конкретен модел трябва да бъде съобразен с бизнес критичността на услугата.

В контекста на устойчивите SaaS решения високата наличност не е просто техническа характеристика, а стратегическо конкурентно предимство. Организациите, които успешно интегрират принципите на устойчивост в своята архитектура, изграждат доверие у потребителите и намаляват дългосрочния оперативен риск.

2.5. Стратегии за възстановяване при аварии

В съвременните облачни среди осигуряването на непрекъсваемост на услугите е критично изискване за всяка SaaS платформа. Независимо от нивото на надеждност на инфраструктурата, възможността за възникване на аварии, хардуерни повреди, софтуерни грешки или мрежови прекъсвания остава реален риск. Поради тази причина организациите трябва да разработят ефективни стратегии за възстановяване при аварии, които да гарантират бързо възстановяване на системите и минимизиране на загубата на данни. Стратегиите за възстановяване при аварии се основават на предварително дефинирани цели за възстановяване. Най-често използваните показатели включват:

- Recovery Time Objective (RTO)
- Recovery Point Objective (RPO)

Тези показатели определят архитектурните решения, които трябва да бъдат внедрени. Например системи с критично значение за бизнеса изискват много ниски стойности на RTO и RPO, което предполага използването на високостепенни механизми за резервираност и репликация.

В практиката се използват няколко основни архитектурни модела за disaster recovery:

- Backup and Restore - данните се архивират периодично и при възникване на авария се възстановяват от резервно копие. Този модел е сравнително евтин, но времето за възстановяване може да бъде значително.
- Pilot Light - основните компоненти на системата се поддържат в минимално работно състояние в резервна среда. При възникване на авария инфраструктурата се разширява и активира пълната функционалност на системата.
- Warm Standby - резервната среда функционира в ограничен мащаб и може сравнително бързо да поеме натоварването при отказ на основната система.
- Active-Active архитектура - две или повече среди функционират едновременно и обслужват потребителски заявки. При отпадане на една среда останалите поемат натоварването без значително прекъсване на услугата.

Облачните платформи предоставят редица механизми, които подпомагат изграждането на ефективни стратегии за възстановяване. Сред тях са:

- автоматична репликация на данни между региони
- географски разпределени центрове за данни
- автоматизирано създаване на резервни копия
- глобални системи за балансиране на натоварването

В този контекст облачни доставчици като Amazon Web Services, Microsoft Azure и Google Cloud Platform предлагат специализирани услуги за репликация на данни и управление на аварийни сценарии. Ефективната стратегия за disaster recovery не се ограничава единствено до техническа инфраструктура.

Тя включва също:

1. ясно дефинирани процедури за реагиране при инциденти
2. редовно тестване на сценарии за възстановяване
3. автоматизирани процеси за възстановяване на инфраструктурата
4. обучение на екипите за работа при аварийни ситуации

Следователно успешното управление на аварии изисква комбинация от архитектурно планиране, автоматизация и организационна подготовка. В контекста на устойчивите SaaS решения ефективните стратегии за възстановяване при аварии гарантират висока степен на надеждност и намаляват риска от дългосрочни прекъсвания на услугата.

2.6. Управление на инфраструктурата чрез код и ролята на Terraform

С нарастването на сложността на облачните инфраструктури традиционните методи за ръчно управление на ресурси стават неефективни и податливи на грешки. В този контекст се утвърждава концепцията за Infrastructure as Code (IaC) - подход, при който инфраструктурата се описва и управлява чрез програмируеми конфигурационни файлове. Infrastructure as Code позволява създаването, конфигурирането и управлението на инфраструктурни ресурси по същия начин, по който се разработва софтуер. [5] Конфигурациите се съхраняват в системи за контрол на версиите, автоматизират се процесите на внедряване и се осигурява възможност за бързо възстановяване на средата при необходимост.

Основните предимства на този подход включват:

- автоматизация на инфраструктурните процеси
- повторяемост и предвидимост на внедряванията
- намаляване на човешките грешки
- лесно възстановяване на инфраструктурата
- по-добра интеграция с DevOps практиките

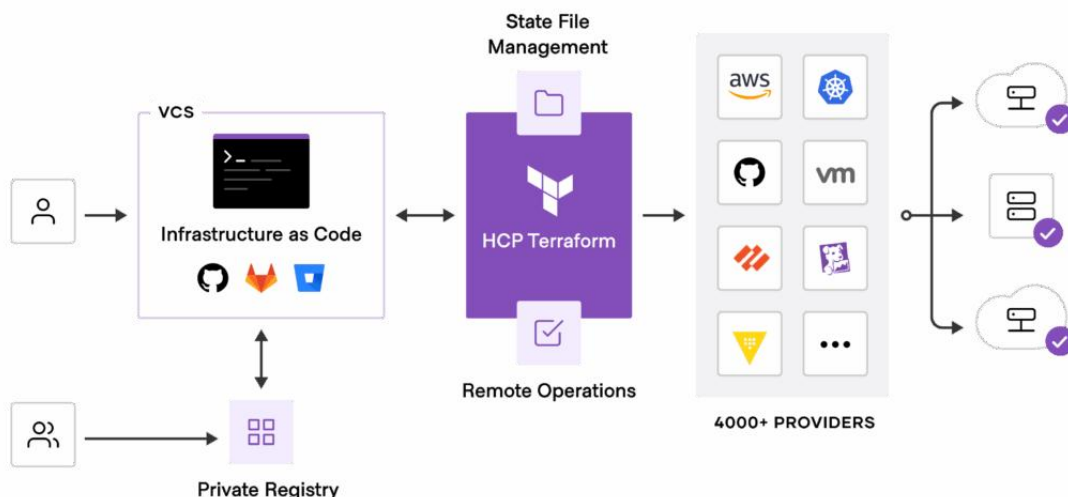
Сред най-популярните инструменти за управление на инфраструктура чрез код е Terraform, разработен от HashiCorp. Terraform позволява описването на инфраструктурни ресурси чрез декларативен конфигурационен език, известен като HashiCorp Configuration Language (HCL).

Основните характеристики на Terraform включват:

- поддръжка на множество облачни платформи
- декларативен модел за управление на инфраструктура
- автоматично управление на зависимости между ресурсите
- възможност за планиране и визуализиране на промените преди тяхното прилагане
- интеграция с системи за автоматизация и CI/CD процеси

Едно от ключовите предимства на Terraform е неговата многооблачна съвместимост. Чрез използването на различни доставчици модули инструментът може да управлява ресурси в платформи като Amazon Web Services, Microsoft Azure и Google Cloud Platform. Това позволява изграждането на унифицирани инфраструктурни модели, независимо от конкретния доставчик. Освен автоматизацията, Terraform подпомага и устойчивостта на системите. При наличие на дефинирани конфигурации инфраструктурата може бързо да бъде възстановена или пресъздадена в нова среда. Това е особено важно при сценарии за disaster recovery и миграция между облачни платформи.

В контекста на съвременните SaaS архитектури Infrastructure as Code се превръща в основен инструмент за управление на сложни разпределени среди. Чрез използването на Terraform организациите могат да изграждат мащабируеми, повторяеми и надеждни инфраструктурни решения, които подкрепят принципите на DevOps, автоматизация и облачна гъвкавост.



- източник: codinic.com

Снимката илюстрира процеса на управление на инфраструктура чрез концепцията Infrastructure as Code с помощта на Terraform, разработен от HashiCorp. В представения модел инфраструктурната конфигурация се съхранява като код в системата за контрол на версиите (VCS). Това позволява проследимост на промените, съвместна работа между екипите и интеграция с автоматизирани процеси за внедряване. Конфигурационните файлове се обработват от Terraform, който анализира желаното състояние на инфраструктурата и създава план за необходимите промени. Инструментът използва т.нар. state файл, който съдържа информация за текущото състояние на инфраструктурните ресурси. След изпълнение на конфигурацията Terraform комуникира с различни облачни и инфраструктурни платформи чрез специализирани модули. Това позволява създаването и управлението на ресурси в среди като Amazon Web Services, Microsoft Azure и Google Cloud Platform, както и в контейнерни платформи като Kubernetes. Този модел позволява изграждането на автоматизирани, повторяеми и мащабируеми инфраструктурни среди. Освен това той подпомага прилагането на DevOps практики и улеснява управлението на сложни многооблачни архитектури.

2.7. Преносимост на приложенията чрез контейнери и оркестрация

Едно от ключовите изисквания към съвременните SaaS архитектури е способността приложенията да бъдат лесно пренасяни между различни инфраструктурни среди. Тази характеристика, известна като преносимост (portability), позволява на организациите да намалят зависимостта от конкретен облачен доставчик и да осигурят по-висока степен на гъвкавост при внедряване и мащабиране на своите системи.

Контейнеризацията се утвърждава като водеща технология за постигане на тази преносимост. Контейнерите представляват леки, изолирани среди за изпълнение на приложения, които съдържат всички необходими зависимости, библиотеки и конфигурации. Това гарантира, че дадено приложение може да функционира по еднакъв начин независимо от инфраструктурата, върху която се изпълнява. Сред най-широко използваните технологии за контейнеризация е Docker. Той позволява разработчиците да пакетират приложенията си в стандартизирани контейнерни образи, които могат да бъдат стартирани във всяка среда, поддържаща Docker runtime. Това значително намалява проблема „работи на моя компютър“, който често възниква при традиционните методи за внедряване.

С увеличаването на броя на контейнерите възниква необходимост от инструменти за тяхното управление и координация. Тук се появява концепцията за оркестрация на контейнери. Най-разпространената платформа за тази цел е Kubernetes, която автоматизира разполагането, мащабирането и управлението на контейнеризирани приложения.

Kubernetes предоставя редица механизми, които подпомагат изграждането на устойчиви и мащабируеми системи, включително:

- автоматично разполагане на контейнери върху клъстер от сървъри
- динамично хоризонтално мащабиране на услугите
- автоматично рестартиране на отказали компоненти
- управление на конфигурации и секрети
- вътрешно балансиране на натоварването между услугите

Тези възможности позволяват изграждането на гъвкави архитектури, при които отделните компоненти на системата могат да бъдат независимо обновявани и мащабирани. Това е особено важно при микросървисните архитектури, където приложението се състои от множество независими услуги. Контейнеризацията също така подпомага прилагането на многооблачни стратегии. Контейнерните образи могат да бъдат изпълнявани в различни инфраструктурни среди, включително платформи като Amazon Web Services, Microsoft Azure и Google Cloud Platform. Това позволява на организациите да изграждат cloud-agnostic архитектури и да избегнат силна зависимост от конкретен доставчик.

В контекста на устойчивите SaaS решения комбинацията от контейнеризация и оркестрация предоставя мощен механизъм за управление на сложни разпределени системи. Чрез

стандартизиране на средата за изпълнение и автоматизиране на процесите по управление на инфраструктурата, тези технологии допринасят за по-висока надеждност, мащабируемост и гъвкавост на софтуерните платформи.

2.8. Съвременни тенденции и примери от практиката

Развитието на облачните технологии и разпределените архитектури води до появата на нови тенденции в проектирането и управлението на SaaS системи. Организациите все по-често възприемат архитектурни подходи, които комбинират автоматизация, мащабируемост и устойчивост, за да отговорят на динамичните изисквания на глобалния дигитален пазар. Една от най-значимите тенденции е широкото прилагане на микросървисни архитектури. При този модел приложенията се изграждат като набор от малки, независими услуги, които комуникират чрез API интерфейси. Този подход позволява по-гъвкаво развитие на системите и улеснява внедряването на нови функционалности без необходимост от промяна на цялата архитектура.

Друга важна тенденция е интеграцията на DevOps практики, които обединяват процесите по разработка и експлоатация на софтуер. Чрез автоматизирани CI/CD процеси организациите могат значително да ускорят цикъла на разработка и внедряване на нови версии на приложенията. Инструменти като Jenkins подпомагат автоматизацията на тези процеси и осигуряват по-висока степен на контрол върху жизнения цикъл на софтуера. Все по-голямо значение придобива и използването на Infrastructure as Code подходи с инструменти като Terraform. Този модел позволява автоматизирано управление на инфраструктурата и осигурява възможност за бързо възстановяване или мащабиране на системите при промяна на натоварването.

В практиката редица глобални технологични компании използват подобни архитектурни подходи за изграждане на мащабируеми SaaS платформи. Например компании като Netflix и Spotify изграждат своите системи върху микросървисни архитектури и силно автоматизирани инфраструктурни среди. Това им позволява да обслужват милиони потребители по света, като същевременно поддържат висока степен на наличност и гъвкавост. Наблюдава се и засилен интерес към многооблачни и хибридни архитектури, които позволяват комбиниране на различни облачни платформи в рамките на една система. Този подход дава възможност за

оптимизиране на разходите, повишаване на устойчивостта и намаляване на риска от зависимост от един доставчик.

В заключение, съвременните тенденции в развитието на SaaS архитектурите са насочени към изграждане на гъвкави, автоматизирани и устойчиви системи. Комбинацията от контейнеризация, микросървисни архитектури, DevOps практики и Infrastructure as Code позволява на организациите да създават платформи, които могат ефективно да отговорят на бързо променящите се технологични и бизнес изисквания.

ГЛАВА 3. БИЗНЕС АНАЛИЗ, РЕГУЛАЦИИ И ПРИЛОЖИМОСТ

3.1. Икономически аспекти и управление на разходите

С нарастващото навлизане на облачните технологии в организациите, икономическите аспекти на тяхното използване придобиват ключово значение. Облачните изчисления не само трансформират начина, по който се изграждат и управляват информационните системи, но и фундаментално променят финансовия модел на организациите.

В традиционните IT среди инвестициите в инфраструктура са свързани със значителни първоначални разходи, включително закупуване на сървъри, мрежово оборудване, лицензи за софтуер и изграждане на центрове за данни. Този модел изисква дългосрочно планиране и често води до неефективно използване на ресурсите. За разлика от него, облачните услуги предоставят възможност за динамично използване на ресурси, при което организациите заплащат само за реално консумираните услуги.

Един от основните икономически принципи при облачните технологии е моделът „плащане според потреблението“ (pay-as-you-go). Той позволява оптимизация на разходите чрез гъвкаво управление на ресурсите и избягване на ненужни инвестиции. В този контекст организациите могат да адаптират своите разходи спрямо текущото натоварване и бизнес нуждите.

Управлението на разходите в облачна среда обаче изисква нов подход и специализирани практики. Въпреки че облачният модел предоставя гъвкавост, той може да доведе до неконтролируемо нарастване на разходите при липса на ефективно управление.

Основните фактори, влияещи върху разходите, включват:

- използваните изчислителни ресурси (виртуални машини, контейнери)
- обемът на съхраняваните данни
- мрежовият трафик и трансфер на данни
- използването на допълнителни услуги (бази данни, AI, аналитични инструменти)

Ефективното управление на разходите включва прилагането на редица добри практики:

- мониторинг и анализ на потреблението в реално време
- автоматично изключване на неизползвани ресурси
- използване на резервирани инстанции или дългосрочни договори
- оптимизация на архитектурата с цел намаляване на излишните ресурси

В този контекст се появява и концепцията FinOps (Financial Operations), която представлява съвкупност от практики за управление и оптимизация на разходите в облачна среда. FinOps обединява финансовите, техническите и бизнес екипите с цел постигане на максимална ефективност при използването на облачни ресурси. [6]

Важен аспект на икономическия анализ е и оценката на възвръщаемостта на инвестицията (ROI). Облачните технологии позволяват по-бързо внедряване на нови услуги, което може да доведе до по-бързо генериране на приходи. Освен това те намаляват времето за излизане на пазара (time-to-market), което е критичен фактор в конкурентна среда.

В рамките на многооблачни архитектури управлението на разходите става още по-комплексно. Различните доставчици, като Amazon Web Services, Google Cloud Platform и Microsoft Azure, използват различни модели на ценообразуване. Това изисква внимателен анализ и стратегическо разпределение на натоварването между тях с цел оптимизация на разходите.

В заключение, икономическите аспекти на облачните технологии изискват интегриран подход, който съчетава технически знания, финансов анализ и стратегическо планиране. Ефективното управление на разходите е ключов фактор за успешното внедряване и устойчивото развитие на SaaS решенията.

3.2. Капиталови и оперативни разходи

Един от най-съществените аспекти при преминаването към облачни технологии е трансформацията на финансовия модел на организациите от капиталови разходи към оперативни разходи. Тази промяна оказва значително влияние върху начина, по който се планират, управляват и оптимизират ИТ инвестициите.

Капиталовите разходи (CAPEX) представляват инвестиции в дълготрайни активи, като сървъри, мрежово оборудване и центрове за данни. Тези разходи се извършват предварително и се амортизират за определен период от време.

Основните характеристики на CAPEX включват:

- високи първоначални инвестиции
- дългосрочно планиране и обвързаност
- риск от остаряване на технологиите
- необходимост от поддръжка и обновяване

От друга страна, оперативните разходи (OPEX) представляват текущи разходи за използване на услуги. В облачния модел организациите не притежават инфраструктурата, а я използват като услуга.

Това води до:

- липса на значителни първоначални инвестиции
- гъвкавост при управление на бюджета
- възможност за плащане според реалното потребление
- по-добра адаптация към динамични бизнес нужди

Преходът от CAPEX към OPEX позволява на организациите да намалят финансовия риск и да освободят капитал за други стратегически инвестиции. Вместо да инвестират в инфраструктура, те могат да насочат ресурсите си към развитие на продукти, маркетинг и иновации. Въпреки предимствата на OPEX модела, той също има своите предизвикателства. Един от основните рискове е възможността за неконтролируемо нарастване на разходите при неправилно управление на ресурсите. Липсата на ясна стратегия и мониторинг може да доведе до значителни финансови загуби.

Сравнението между двата модела може да бъде обобщено по следния начин:

- CAPEX е подходящ при стабилни и предвидими натоварвания
- OPEX е по-ефективен при динамични и променливи бизнес условия
- CAPEX осигурява по-голям контрол върху инфраструктурата
- OPEX предоставя по-голяма гъвкавост и мащабируемост

В практиката много организации използват хибриден подход, при който комбинират двата модела. Например критични системи могат да се поддържат локално (CAPEX), докато динамични услуги се разполагат в облака (OPEX). В контекста на SaaS решенията оперативният модел доминира, тъй като той позволява бързо внедряване, лесно мащабиране и по-добра икономическа ефективност. Това го прави предпочитан избор за съвременните организации, стремящи се към дигитална трансформация и устойчив растеж.

3.3. Анализ на целевите групи организации

Облачните технологии и SaaS моделът намират широко приложение в организации с различен мащаб и структура. Въпреки това, нуждите, приоритетите и предизвикателствата при внедряването им се различават значително в зависимост от размера на организацията, индустрията и нивото на дигитална зрялост.

Анализът на целевите групи организации позволява по-добро разбиране на факторите, които влияят върху избора на архитектурни решения, както и на бизнес стойността, която облачните технологии предоставят.

3.3.1. Малки и средни предприятия - гъвкавост и оптимизация на ресурси

Малките и средни предприятия (МСП) представляват значителна част от икономиката и често са изправени пред ограничения, свързани с финансови ресурси, технически капацитет и експертиза. За тях облачните технологии предлагат възможност за достъп до високотехнологични решения без необходимост от значителни първоначални инвестиции. Една от основните ползи за МСП е гъвкавостта, която облачният модел предоставя. Организациите могат бързо да внедряват нови услуги, да експериментират с различни бизнес

модели и да адаптират своите системи спрямо пазарните условия. Това е особено важно в динамична конкурентна среда, където скоростта на реакция е ключов фактор за успех. Оптимизацията на ресурсите е друг съществен аспект. Чрез използване на SaaS решения МСП могат да избегнат разходите за поддръжка на собствена инфраструктура и да се фокусират върху основната си дейност.

Това включва:

- намаляване на нуждата от вътрешни IT екипи
- използване на готови решения за бизнес процеси (CRM, ERP, e-commerce)
- автоматизация на рутинни операции

Въпреки тези предимства, МСП се сблъскват и с определени предизвикателства:

- ограничен контрол върху инфраструктурата
- зависимост от доставчици на услуги
- липса на експертиза за управление на сложни облачни среди

В този контекст многооблачните подходи могат да бъдат използвани в по-ограничен мащаб, основно с цел повишаване на надеждността и избягване на пълна зависимост от един доставчик. Въпреки това, сложността на управлението често налага използването на по-опростени архитектури.

3.3.2. Големи предприятия - изисквания за сигурност и мащаб

Големите организации и корпорации имат съществено различни изисквания към своите IT системи. Те обработват големи обеми от данни, обслужват значителен брой потребители и често оперират в силно регулирани индустрии като финанси, здравеопазване и телекомуникации. За тези организации ключовите фактори при внедряване на облачни решения са сигурността, мащабируемостта и устойчивостта. Сигурността е от първостепенно значение, тъй като компрометирането на данни може да доведе до сериозни финансови и репутационни щети.

Големите предприятия прилагат комплексни механизми за защита, включително:

- криптиране на данни при съхранение и пренос
- управление на достъпа и идентичностите (IAM)
- мониторинг и откриване на заплахи в реално време
- сегментация на мрежите и изолиране на критични системи

Мащабируемостта също е критичен фактор. Облачните платформи позволяват динамично разширяване на ресурсите, което е особено важно при глобални услуги с милиони потребители. Компании от мащаба на Amazon и Google използват разпределени облачни архитектури, за да осигурят непрекъсваемост на услугите и висока производителност. Големите предприятия са и основните потребители на многооблачни и поли-облачни архитектури.

Те използват различни доставчици, като Amazon Web Services, Microsoft Azure и Google Cloud Platform, с цел:

- разпределение на риска
- оптимизация на разходите
- използване на специфични услуги от различни доставчици

Въпреки това, управлението на такива сложни среди изисква високо ниво на експертиза, внедряване на автоматизация и използване на инструменти за централизирано управление.

3.4. Нормативни изисквания и защита на данните (GDPR, DORA)

С нарастващата зависимост на организациите от облачните технологии, въпросите, свързани със защитата на данните и регулаторното съответствие, придобиват критично значение. Европейският съюз въвежда редица регулации, които поставят строги изисквания към обработката, съхранението и управлението на данни в дигитална среда.

Един от най-важните регулаторни актове е General Data Protection Regulation (GDPR). Той определя рамката за защита на личните данни и налага задължения на организациите, които обработват информация за граждани на ЕС.

Основните принципи на GDPR включват:

- законосъобразност, добросъвестност и прозрачност
- ограничаване на целите на обработката
- минимизация на данните
- точност и актуалност
- ограничение на съхранението
- интегритет и поверителност

В контекста на облачните технологии спазването на GDPR изисква внимателен избор на доставчици и архитектурни решения.

Организациите трябва да гарантират, че:

- данните се съхраняват в съответствие с изискванията за географска локализация
- се прилагат механизми за криптиране и контрол на достъпа
- има ясни договорни отношения с доставчиците относно обработката на данни

Друг ключов регулаторен акт е Digital Operational Resilience Act (DORA), който е насочен основно към финансовия сектор. Той поставя изисквания за дигитална оперативна устойчивост и управление на риска, свързан с информационните и комуникационни технологии.

DORA изисква от организациите:

- изграждане на устойчиви IT системи
- провеждане на тестове за устойчивост (stress testing)
- управление на риска от външни доставчици
- наличие на планове за възстановяване при аварии

Тази регулация има пряко отношение към използването на облачни услуги, тъй като организациите трябва да демонстрират контрол и прозрачност при използване на външни доставчици.

В допълнение към GDPR и DORA, организациите трябва да се съобразяват и с концепцията за дигитален суверенитет, която изисква контрол върху данните и инфраструктурата. Това често

води до използване на хибридни или многооблачни архитектури, при които чувствителните данни се съхраняват в определени географски региони.

В заключение, регулаторните изисквания играят ключова роля при проектирането на съвременни SaaS архитектури. Те не само налагат ограничения, но и стимулират изграждането на по-сигурни, устойчиви и прозрачни системи. Организациите, които успешно интегрират тези изисквания в своите архитектурни решения, могат да постигнат по-високо ниво на доверие от страна на клиентите и конкурентно предимство на пазара.

3.5. Анализ на риска и ограничения на избрания подход

Внедряването на многооблачни и поли-облачни архитектури в контекста на SaaS решенията предоставя значителни предимства по отношение на устойчивост, гъвкавост и независимост от доставчик. Въпреки това, този подход е свързан с редица рискове и ограничения, които следва да бъдат внимателно анализирани и управлявани.

Един от основните рискове е свързан със сложността на архитектурата. Разпределянето на инфраструктурата между множество облачни доставчици води до увеличаване на броя на компонентите, комуникационните зависимости и точките на интеграция. Това от своя страна усложнява управлението, мониторинга и поддръжката на системата. В подобни среди дори малки конфигурационни грешки могат да доведат до значителни проблеми, включително прекъсвания на услуги или изтичане на данни.

Друг съществен риск е свързан с интеграцията между различните платформи. Всеки облачен доставчик, като Amazon Web Services и Google Cloud Platform, използва специфични интерфейси, услуги и механизми за управление.

Това може да доведе до:

- несъвместимости между услугите
- усложнена комуникация между компонентите
- необходимост от допълнителни адаптационни слоеве

Въпреки че целта на многооблачния подход е да намали зависимостта от доставчик, на практика може да възникне т.нар. „скрита зависимост“. Това се случва, когато дадена архитектура разчита на специфични услуги или функционалности, които трудно могат да бъдат заменени с алтернативи от друг доставчик.

Финансовият риск също не бива да бъде подценяван. Управлението на ресурси в повече от една облачна среда изисква внимателно планиране и мониторинг.

Без ефективна стратегия за управление на разходите е възможно:

- дублиране на ресурси
- неефективно използване на инфраструктурата
- увеличаване на оперативните разходи

Допълнително предизвикателство представлява необходимостта от високо ниво на експертиза. Управлението на многооблачни среди изисква знания в различни платформи, инструменти и технологии.

Това може да доведе до:

- повишени разходи за обучение и квалификация
- зависимост от ограничен брой специалисти
- по-трудно управление на екипи и процеси

От гледна точка на сигурността, разпределените архитектури увеличават повърхността за атаки. Всеки допълнителен компонент или интеграционна точка представлява потенциална уязвимост.

Основните рискове включват:

- неправилна конфигурация на достъпите
- липса на унифицирана политика за сигурност
- трудности при централизирано наблюдение и реакция при инциденти

Регулаторните ограничения също оказват влияние върху избора на архитектура. Изискванията за защита на данните, като тези, заложи в General Data Protection Regulation, могат да наложат ограничения върху това къде и как се съхраняват данните. Това може да ограничи използването на определени облачни услуги или да изисква допълнителни мерки за съответствие. Не на последно място, трябва да се отчете и рискът, свързан с производителността и латентността.

Комуникацията между различни облачни среди може да доведе до:

- увеличено време за отговор
- зависимост от мрежова свързаност
- потенциални тесни места (bottlenecks) в системата

Въпреки изброените рискове, много от тях могат да бъдат ефективно управлявани чрез прилагане на добри практики, включително:

- използване на стандартизирани технологии (контейнери, API)
- автоматизация чрез Infrastructure as Code
- централизирано наблюдение и логиране
- прилагане на принципите на Zero Trust сигурност

Многооблачният подход не представлява универсално решение, а стратегически избор, който трябва да бъде съобразен с конкретните нужди, ресурси и цели на организацията. Внимателният анализ на рисковете и ограниченията е ключов за изграждането на устойчива и ефективна SaaS архитектура.

ГЛАВА 4. ПРОЕКТИРАНЕ НА РЕФЕРЕНТНА СОФТУЕР КАТО УСЛУГА АРХИТЕКТУРА ЗА ПРИМЕРНА ОРГАНИЗАЦИЯ

4.1. Цели и обхват на архитектурата

Дефиниране на примерната организация (Бизнес контекст).

За целите на настоящата магистърска теза и за да се демонстрира практическата приложимост на теоретичните концепции, се въвежда референтна B2B (Business-to-Business) технологична компания, условно наречена „FinCloud Analytics“. Организацията оперира във финансово-технологичния (FinTech) сектор и разработва едноименна "Софтуер като услуга" (SaaS) платформа. Основната функционалност на платформата включва електронно фактуриране, обработка на масови финансови документи, генериране на счетоводни отчети и предоставяне на анализи в реално време за средни и големи корпоративни клиенти.

Поради естеството на обработваните данни, SaaS платформата „FinCloud Analytics“ се категоризира като критична за бизнеса (mission-critical) система. Всяко непредвидено прекъсване на услугата (downtime), особено в края на отчетни финансови периоди, води до невъзможност на клиентите да оперират със своите средства. Това генерира директни финансови загуби, нарушава строгите споразумения за гарантирано ниво на обслужване (Service Level Agreements - SLA) и нанася непоправими репутационни щети на компанията.

Следователно, основният бизнес проблем, който организацията трябва да разреши чрез инфраструктурна стратегия, е пълното елиминиране на зависимостта от единична точка на отказ (Single Point of Failure - SPOF) на ниво облачен доставчик.

Архитектурни цели: За да се отговори на гореописаното бизнес предизвикателство, основната цел на проекта е да се проектира, дефинира и реализира активно-пасивна (Active-Passive) многооблачна архитектура с интегриран механизъм за автоматизирано превключване при отказ (Failover).

Специфичните архитектурни цели включват:

1. **Гарантирана непрекъсваемост (High Availability):** Осигуряване на механизъм, при който потребителският трафик се пренасочва мигновено към независим резервен облачен център в случай на глобален или регионален срив в основния доставчик.
2. **Инфраструктурна гъвкавост и повторяемост:** Изграждане на средата по начин, който позволява нейното бързо разгръщане, модифициране и унищожаване чрез декларативен код (Infrastructure as Code), минимизирайки риска от човешки грешки.
3. **Икономическа ефективност (ОРЕХ оптимизация):** Проектиране на архитектурата като "топъл резерв" (Warm Standby), при който резервната среда консумира минимални изчислителни ресурси, докато не бъде активирана, като по този начин се контролират оперативните разходи.

Обхват и ограничения на проекта: За да се запази фокусът на изследването върху инфраструктурната устойчивост, обхватът на референтната архитектура е строго дефиниран:

- **Включено в обхвата:** Проектиране на външния маршрутизиращ слой (Global DNS routing), изчислителния слой (Compute instances) за изпълнение на приложната логика на "FinCloud Analytics", мрежовия слой (Virtual Private Clouds) и правилата за сигурност на мрежово ниво.

- **Исключено от обхвата:** Настоящата разработка не включва писане на сложен софтуерен код за самите финансови алгоритми на SaaS платформата, нито изграждането на сложни процеси за непрекъсната интеграция и доставка (CI/CD pipelines). Приложението ще бъде абстрахирано чрез стандартизиран уеб сървър, който играе ролята на потребителския интерфейс.

4.2. Функционални и нефункционални изисквания

Успешното реализиране на архитектурата изисква предварителното дефиниране на строги параметри, на които системата трябва да отговаря, за да обслужва адекватно клиентите на „FinCloud Analytics“.

Функционални изисквания: Функционалните изисквания описват какво трябва да прави системата от гледна точка на крайния потребител и оперативното поведение:

- **Уеб-базиран достъп:** Системата трябва да предоставя непрекъснат достъп до потребителския интерфейс (Frontend) чрез стандартни уеб протоколи (HTTP/HTTPS), като отговаря на клиентските заявки без видими забавяния.
- **Обслужване на процеси:** Инфраструктурата трябва да поддържа изпълнението на уеб сървърни процеси (например Nginx или Apache), които да визуализират състоянието на платформата и да обработват входящия трафик.
- **Изоляция на средата:** Ресурсите, обслужващи платформата, трябва да бъдат логически изолирани в собствени виртуални мрежи, за да се предотврати неоторизиран вътрешен достъп.

Нефункционални изисквания: Нефункционалните изисквания са критични за дизайна на многооблачната среда, тъй като те диктуват избора на конкретни технологии и стратегии за възстановяване:

1. **Целево време за възстановяване (Recovery Time Objective - RTO):** Това е най-критичната метрика за проекта. Времето за пълно превключване на потребителския трафик от основния към резервния облачен доставчик при възникване на мащабна авария трябва да бъде сведено до минимум. Заложената цел е $RTO < 3$ минути. (това е заради ползването на максимално евтините планове от доставчиците AWS и GCP, при реална разработка и клиенти, тези цифри могат драстично да спаднат.)

2. **Глобална наличност (High Availability):** Архитектурата трябва да бъде проектирана така, че да подsigури теоретична наличност от 99.99%. Това налага използването на минимум два напълно независими центъра за данни, управлявани от различни корпоративни субекти (за избягване на системен срив при един доставчик).
3. **Технологична преносимост (Portability):** Приложната логика на "FinCloud Analytics" не трябва да бъде тясно обвързана със специфични IaaS (Infrastructure as a Service) услуги на конкретен облак (например използване на специфични AWS функции). Системата трябва да може да функционира абсолютно идентично, независимо дали се изпълнява върху сървъри на Amazon, Google или локален център за данни.
4. **Сигурност на комуникацията:** Целият външен трафик между крайния потребител и инфраструктурата, както и проверките за състоянието (Health Checks) между облаците, трябва да бъдат защитени чрез рестриктивни правила на защитните стени (Security Groups/Firewall rules), допускащи трафик само по строго определени портове (напр. порт 80 за уеб трафик и порт 22 за административен SSH достъп от определени IP адреси).

4.3. Обосновка на избора на технологии и доставчици

Изграждането на многооблачна архитектура за критична финансова система като "FinCloud Analytics" изисква прецизен подбор на технологичния стек. Всяко решение трябва да бъде обосновано не само технически, но и стратегически, за да се гарантира баланс между надеждност, цена и лекота на управление.

1. Избор на инфраструктурни доставчици (IaaS Providers) За реализацията на архитектурата са избрани два от водещите глобални доставчици на публични облачни услуги:

- **Amazon Web Services (AWS) - Основна среда (Primary Site):** AWS е избран за хостване на активната производствена среда поради своята лидерска позиция на пазара (най-голям пазарен дял), изключително зрялата си инфраструктура и високото ниво на сигурност. За изпълнение на приложната логика ще се използват виртуални машини (Amazon EC2 - Elastic Compute Cloud).
- **Google Cloud Platform (GCP) - Резервна среда (Secondary / DR Site):** За да се елиминира рискът от технологична зависимост (Vendor Lock-in) и да се подsigури системата срещу мащабни сринове в мрежата на Amazon, като втори независим доставчик е избран GCP. Платформата на Google се отличава със своята високоскоростна глобална оптична мрежа (Premium Network Tier) и гъвкави ценови модели за

изчислителни ресурси (Google Compute Engine). Комбинацията от AWS и GCP гарантира, че дори при отпадане на един от гигантите, бизнесът на "FinCloud Analytics" ще продължи да функционира.

2. Инструмент за глобално маршрутизиране (Global DNS & Traffic

Management) Управлението на трафика между два независими облака изисква специализирано решение. В процеса на проектиране бяха разгледани алтернативи като използване на локален DNS сървър или вградените инструменти на GCP (Cloud DNS).

- *Ограничения на алтернативите:* Локалният (вътрешнофирмен) DNS е неприложим, защото като SaaS платформата трябва да обслужва публичен интернет трафик от външни клиенти. От друга страна, конфигурирането на междублачни "Health Checks" (проверки за състоянието) и автоматично превключване при отказ (Failover routing) чрез GCP Cloud DNS изисква по-сложна интеграция с допълнителни услуги за балансиране на натоварването.
- Избрано решение - **AWS Route 53:** AWS Route 53 се е утвърдил като индустриален стандарт за глобално маршрутизиране. Той предлага вградена, лесна за конфигуриране политика за маршрутизиране при отказ (Failover Routing Policy). Route 53 позволява създаването на (Health Checks), които могат да следят състоянието на ресурси както вътре в AWS, така и извън него (в нашия случай - в GCP). Това го прави идеалният "арбитър", който да пренасочва потребителския трафик мигновено при регистриране на авария.

3. Инструмент за оркестрация и инфраструктура като код (IaC)

- Избрано решение - **HashiCorp Terraform:** Докато AWS и GCP предлагат собствени инструменти за инфраструктура като код (съответно AWS CloudFormation и Google Cloud Deployment Manager), тяхното използване би ограничило екипа само до една екосистема. Terraform е избран поради своята абсолютна агностичност към доставчика (Cloud-Agnostic). Чрез своя декларативен език (HCL - HashiCorp Configuration Language) и система от модули (Providers), Terraform позволява цялостната архитектура - мрежи, защитни стени и виртуални машини в AWS и GCP - да бъде описана, версионизирана и разгърната от една единствена кодова база. [7]

4. Изчислителна среда и уеб сървър

- За да се гарантира, че потребителският интерфейс на платформата ще работи абсолютно идентично в двата облака, се използва стандартизирана операционна система (Linux Ubuntu/Debian) и утвърден, лек уеб сървър (**Nginx**). Nginx е избран заради своята висока производителност при обслужване на статично съдържание и възможностите си за работа като обратен прокси сървър (Reverse Proxy), което го прави стандарт при изграждането на микросървисни и облачни архитектури.

4.4. Общ архитектурен модел на решението

Общият архитектурен модел, проектиран за нуждите на „FinCloud Analytics“, е базиран на концепцията **Active-Passive (Активен-Пасивен) с Топъл резерв (Warm Standby)** в многооблачна конфигурация. Този модел е стратегически избран, тъй като предоставя отличен баланс между висока степен на надеждност (High Availability) и контрол върху оперативните разходи (OPEX), избягвайки финансовата тежест на двойна Active-Active инфраструктура.

Архитектурата функционира по следния логически модел, състоящ се от две паралелни среди и един контролен слой:

1. Контролен слой (The Traffic Controller): На най-високо ниво се намира глобалният DNS маршрутизатор (AWS Route 53). Той не обработва самите финансови данни на клиентите, а действа като „интелигентен диспечер“. В него са конфигурирани адресите и на двете среди (AWS и GCP), но е зададен ясен приоритет: 100% от нормалния потребителски трафик се насочва към Основната среда.

2. Основна среда - Активна (Primary Active Environment - AWS): Това е производствената среда, която поема цялото натоварване от клиентите на SaaS платформата при нормални експлоатационни условия. Тя се състои от изолирана виртуална частна мрежа (VPC) в европейски регион на AWS, в която функционират изчислителните ресурси (EC2 инстанции), обслужващи уеб интерфейса на "FinCloud Analytics". Тази среда се намира под постоянен мониторинг (Health Checks) от страна на контролния слой.

3. Резервна среда - Пасивна (Secondary Passive Environment - GCP): Това е средата за възстановяване при аварии (Disaster Recovery). Тя е физически и логически напълно независима, разположена в инфраструктурата на Google Cloud. В нея предварително е разгърнато абсолютно точно копие на мрежовата топология, защитните правила и уеб сървърите (чрез автоматизацията на Terraform). Терминът „Пасивна“ означава, че при нормални условия тази среда не приема публичен потребителски трафик. Терминът „Топъл резерв“ (Warm Standby) означава, че сървърите са включени, конфигурирани и готови да заработят на пълни обороти в рамките на секунди, веднага щом контролният слой им пренасочи трафика.

Този модел гарантира, че дори при катастрофално събитие (като например прекъсване на оптичен кабел към център за данни на Amazon или глобален срив в техния контролен панел, също природни бедствия като земетресение, пожар), "FinCloud Analytics" притежава напълно независима, вече работеща и конфигурирана резервна „база“, към която клиентите ще бъдат пренасочени автоматично, минимизирайки времето за прекъсване на бизнес процесите им.

4.5. Компонентна структура и мрежова топология

За да се осигури функционирането на проектирания Active-Passive модел, мрежовата топология на „FinCloud Analytics“ е физически и логически сегментирана в три основни слоя. Всеки слой е дефиниран чрез декларативен код (Terraform) и изпълнява строго определена роля в процеса по обслужване на клиентските заявки и подsigуряване на услугата.

1. Слой за глобално маршрутизиране и мониторинг (Global Routing Layer) Този слой е разположен извън границите на конкретните центрове за данни и функционира на глобално ниво чрез услугата **AWS Route 53**.

Компонентната му структура включва:

- **Hosted Zone (Хоствана зона):** Логически контейнер, който управлява DNS записите за публичния домейн на платформата.
- **Health Checks:** Конфигурирани са автоматизирани агенти, които изпращат HTTP заявки (ping) към основната инфраструктура на всеки 10 до 30 секунди. Тези проверки оценяват не само мрежовата свързаност, но и HTTP кода за отговор (очаквайки статус код 200 OK), за да потвърдят, че уеб сървърът функционира коректно.
- **Failover Routing Policy (Политика за превключване):** Тук са дефинирани два DNS - записа. Основният (Primary) запис сочи към IP адреса на AWS средата и е обвързан с

гореспоменатия Health Check. Резервният (Secondary) запис сочи към IP адреса на GCP средата и се активира единствено при негативен резултат от health check.

2. Основен инфраструктурен сегмент (Primary Site - AWS) Това е ядрото на производствената среда, разположено в избрания регион на Amazon Web Services.

- **Virtual Private Cloud (VPC):** Изолирана виртуална мрежа, която дефинира логическото адресно пространство (CIDR блок).
- **Мрежов достъп (Internet Gateway & Subnets):** Конфигуриран е Internet Gateway, който позволява двупосочна комуникация между публичната подмрежа (Public Subnet) и глобалната интернет мрежа.
- **Групи за сигурност (Security Groups):** Защитна стена на ниво инстанция (stateful firewall), която е конфигурирана на принципа на "нулевото доверие" (Zero Trust). Разрешен е единствено входящ (Inbound) трафик по порт 80 (HTTP) за обслужване на уеб клиентите и порт 22 (SSH) за защитен административен достъп от специфични IP адреси.
- **Изчислителен възел (Compute Instance):** Виртуална машина тип t2.micro (базирана на Amazon Linux 2 или Ubuntu), върху която е провизиран уеб сървър Nginx. Този сървър репрезентира презентационния слой (Frontend) на SaaS платформата.

3. Резервен инфраструктурен сегмент (Disaster Recovery Site - GCP) Резервната среда е разположена в Google Cloud Platform и е проектирана като огледално, но независимо копие на основната логика.

- **VPC Network:** Независима виртуална мрежа в инфраструктурата на Google, със собствени правила за маршрутизиране.
- **Firewall Rules (Защитни стени):** GCP Firewall правилата репликират рестрикциите от AWS, позволявайки достъп само до уеб и административните портове.
- **Изчислителен възел (Compute Engine):** Виртуална машина тип e2-micro, провизирана със същия уеб сървър (Nginx). За целите на доказване на концепцията (Proof of Concept - PoC), презентационният слой в резервната среда съдържа визуални индикатори (например различна цветова схема или текстови маркери), които позволяват на мрежовите администратори веднага да идентифицират, че системата работи в аварийен режим (Disaster Recovery Mode).

4.6. Сценарии за отказ и механизми за устойчивост

Основната цел на референтната многооблачна архитектура е да предпази SaaS платформата от най-тежките сценарии за инфраструктурен отказ. За да се гарантира устойчивостта, архитектурата е проектирана да реагира автоматизирано по предварително дефиниран алгоритъм.

Дефиниране на критичния сценарий (Failure Scenario) Критичният сценарий обхваща събития, при които основната изчислителна среда в AWS става напълно недостъпна за крайните потребители.

Това може да бъде предизвикано от:

- Мащабен хардуерен или мрежов срив в конкретния Availability Zone на AWS.
- Компрометиране или фатална грешка на ниво операционна система на основната EC2 инстанция (Kernel panic).
- Човешка грешка, водеща до неволно изтриване или спиране на производствения сървър.

Механизъм за възстановяване (Failover Algorithm) Процесът по поддържане на непрекъсваемостта на услугата преминава през следните автоматизирани фази:

1. **Фаза на нормална експлоатация:** При нормални условия, DNS заявките на клиентите на „FinCloud Analytics“ се насочват към основната среда в AWS.
2. **Инцидент и Деградация:** Настъпва един от гореописаните критични сценарии. Уеб сървърът в AWS спира да обслужва заявки.
3. **Фаза на детекция (Health Check Failure):** Route 53, които изпращат периодични HTTP заявки (на всеки 10 секунди), не получават отговор 200 OK. При регистриране на три последователни неуспешни опита (в рамките на 30 секунди), Route 53 категоризира основния AWS възел като "Неактивен" (Unhealthy).
4. **Фаза на превключване (DNS Failover Routing):** След детекцията на срива, Route 53 активира своята Failover политика. Основният запис се спира, а резервният запис (сочещ към статичния IP адрес на GCP инстанцията) става активен.
5. **Фаза на разпространение (Propagation):** Времето за пълно превключване зависи от стойността TTL (Time to Live) на DNS записите. Тъй като архитектурата е оптимизирана за бързо възстановяване, TTL стойността е настроена на 60 секунди, което принуждава

браузърите на потребителите и междинните доставчици (ISP) бързо да обновят своя кеш и да получат новия (GCP) адрес.

6. **Фаза на възстановяване на услугата:** Крайните клиенти, които презаредят своята сесия, биват насочени към резервния инфраструктурен сегмент в GCP. Услугата е напълно възстановена, като заложеното целево време (RTO) от под 3 минути е успешно постигнато без необходимост от ръчна човешка намеса (Zero-touch failover).

4.7. Управление на достъпа и идентичността

Проектирането на многооблачна SaaS архитектура разширява значително повърхността за потенциални кибератаки. Следователно, сигурността не може да бъде разглеждана като вторична задача, а трябва да бъде интегрирана на фундаментално ниво чрез строги политики за управление на идентичността и достъпа (Identity and Access Management - IAM).

Основното предизвикателство при използването на подход "Инфраструктура като код" (IaC) с Terraform е необходимостта от предоставяне на програмни права за създаване, модифициране и изтриване на ресурси едновременно в Amazon Web Services (AWS) и Google Cloud Platform (GCP). За да се минимизира рискът от неоторизиран достъп или компрометиране на системата, архитектурата стриктно следва **Принципа на най-малките привилегии (Principle of Least Privilege - PoLP)**. Този принцип гласи, че на всеки потребител, програма или процес трябва да бъдат предоставени единствено минималните права, които са абсолютно необходими за изпълнение на неговата легитимна функция.

Специфика при автентикацията в AWS (Amazon Web Services) В средата на AWS категорично не се използват главни (Root) акаунти за оперативно управление или оркестрация. Вместо това, в IAM конзолата се създава специализиран потребител (IAM User), предназначен изключително за нуждите на Terraform оркестрацията. За удостоверяване пред API интерфейсите на Amazon се генерират дълготрайни програмни ключове (Access Key ID и Secret Access Key). Вместо предоставяне на глобални административни права (AdministratorAccess), на този специализиран потребител се прикачват IAM политики (Policies). Те ограничават обсега на действие единствено до необходимите услуги за реализация на прототипа: управление на виртуални машини (EC2), мрежови конфигурации (VPC) и управление на DNS записи (Route 53).

Специфика при автентикацията в GCP (Google Cloud Platform) В екосистемата на Google Cloud, най-добрата практика за автоматизация от тип "машина-към-машина" (Machine-to-Machine) налага използването на Сервизни акаунти (Service Accounts), а не на персонални потребителски профили. За целите на прототипната система се създава изолиран сервизен акаунт в конкретния GCP проект. На този акаунт се делегират точни роли (IAM Roles), като например Compute Admin (за създаване и управление на виртуалните инстанции) и Network Admin (за конфигурация на защитните стени). Удостоверяването на Terraform пред Google Cloud API се осъществява чрез криптиран JSON ключ (Service Account Key), който се сваля и съхранява сигурно в локалната среда за разработка.

Управление на конфигурационните тайни (Secrets Management) Критичен аспект от сигурността на многооблачната IaC архитектура е предпазването на автентикационните ключове и конфигурационните файлове.

За да се предотврати потенциално изтичане на чувствителна информация:

1. **Защита на ключовете:** Програмните ключове за AWS и JSON файлът за GCP никога не се записват директно в изходния код (hardcoding). Вместо това, те се подават към Terraform динамично чрез локални променливи на средата (Environment Variables) на компютъра на администратора.
2. **Защита на състоянието (State File):** Terraform генерира файл за състоянието (terraform.tfstate), който съдържа детайлна метаинформация за цялата разгърната инфраструктура (включително публични IP адреси, идентификатори на ресурси и топология). Този файл се третира като строго конфиденциален документ и не се качва в публични системи за контрол на версиите (като GitHub или GitLab).

Чрез стриктното прилагане на тези IAM практики, референтната архитектура на „FinCloud Analytics“ гарантира, че процесът по автоматизирано провизиране на ресурси е не само гъвкав и бърз, но и напълно защитен от външни заплахи.

ГЛАВА 5. РЕАЛИЗАЦИЯ НА ПРОТОТИПНА СИСТЕМА

5.1. Подход към реализацията и методология

За преминаването от концептуален модел към реално функционираща прототипна система е възприет утвърденият в съвременното софтуерно инженерство подход „Инфраструктура като код“ (Infrastructure as Code - IaC). Традиционният метод за управление на инфраструктура чрез ръчно конфигуриране в графични потребителски интерфейси (т.нар. "ClickOps") е напълно отхвърлен поради своята податливост на човешки грешки и невъзможност за проследяване на промените. Вместо това, цялостната многооблачна архитектура е дефинирана чрез декларативен програмен код, използвайки инструмента HashiCorp Terraform.

Този подход осигурява три критични предимства за проекта:

- Идемпотентност: Гаранция, че независимо колко пъти се изпълнява кодът, крайният резултат винаги ще бъде еднаква, точно дефинирана инфраструктурна среда.
- Многооблачна съвместимост (Cloud-Agnosticism): Terraform е един от малкото инструменти, които позволяват управлението на ресурси едновременно в Amazon Web Services (AWS) и Google Cloud Platform (GCP) от една и съща кодова база.
- Версионизиране и проследимост: Инфраструктурата се третира като софтуерен продукт, чиито промени се съхраняват и проследяват във времето.
- Контекст и Проблем: Прототипната SaaS платформа изисква единна точка за достъп (публичен домейн), която да може динамично, надеждно и напълно автономно да пренасочва потребителския трафик между два конкурентни облачни доставчика (AWS и GCP) в случай на авария.
- Разгледани алтернативи: Локален DNS сървър: Отхвърлен, тъй като не предлага глобална наличност и изисква ръчна поддръжка.

- Google Cloud DNS: Отхвърлен, тъй като конфигурацията на междуоблачни проверки за състоянието (Health Checks) изисква по-сложна интеграция с външни балансатори на натоварването.
- Взето решение: Закупен е външен къстъм домейн (fin-cloud.click) чрез глобалния регистратор GoDaddy. Управлението на именните сървъри (Name Servers) е делегирано изцяло към AWS Route 53.

Обосновка (Последствия): AWS Route 53 е избран като "глобален арбитър", тъй като предлага вградени, високо надеждни политики за превключване при отказ (Failover Routing Policy). Най-силното му предимство в конкретния случай е способността на неговите здравни сонди (Health Checks) да следят състоянието не само на ресурси вътре в мрежата на Amazon, но и на външни публични крайни точки (Endpoints), каквато е нашата резервна виртуална машина в инфраструктурата на Google Cloud.

Концептуална аналогия (За лесно асимилиране на логиката):

Можем да си представим DNS системата (AWS Route 53) като изключително интелигентен “Диспечер на железопътен възел”, а домейна fin-cloud.click като билета на пътниците (клиентите на SaaS платформата).

При нормални условия, диспечерът насочва всички влакове с пътници по Главната високоскоростна линия (AWS). Диспечерът обаче има агенти (Health Checks), които на всеки 30 секунди проверяват дали Главната линия е здрава. Ако агентът докладва, че на Главната линия има паднало дърво (срив в сървър), диспечерът не спира движението. Той мигновено и автоматично превключва стрелките на релсите и насочва следващите влакове по предварително изграден, напълно независим Обходен маршрут (GCP). Пътниците стигат до крайната си цел, без дори да разберат, че е имало инцидент на главния път. Чрез инфраструктурата като код, ние сме построили както релсите, така и сме програмирали логиката на диспечера.

5.2. Конфигуриране на облачните среди и мрежова изолация

След дефинирането на глобалния маршрутизиращ слой, реализацията на прототипа преминава към изграждането на мрежовия фундамент. В контекста на съвременните облачни инфраструктури, сигурността и мрежовата изолация са от критично значение, тъй като моделът

на споделяната отговорност (Shared Responsibility Model) изисква от организацията да защити конфигурациите и данните в облака. За да се гарантира максимална защита на SaaS платформата FinCloud Analytics, мрежовата архитектура е изградена при стриктно спазване на принципа за - Нулево доверие (Zero Trust). Това означава, че по подразбиране никакъв мрежови трафик не се допуска до изчислителните ресурси, освен ако не е изрично деклариран и разрешен на ниво код.

Логическа мрежова сегментация (Virtual Private Clouds)

И в двете облачни среди (AWS и GCP) изчислителните машини не се разполагат директно в публичното интернет пространство. Вместо това се създават Виртуални частни мрежи (Virtual Private Clouds - VPC). VPC представлява логически изолиран сегмент от мрежата на доставчика, върху който организацията има пълен контрол - от дефинирането на IP диапазоните (CIDR блокове) до маршрутизирането на трафика към външни портали (Internet Gateways). Разполагането на ресурсите в напълно независими VPC мрежи в два различни облака елиминира риска от странично придвижване (lateral movement) на потенциални кибератаки между платформите.

Прилагане на защитни стени (Security Groups и Firewall Rules) чрез Terraform

Синтаксисът и философията за управление на защитните стени се различават при двата доставчика, но чрез използването на Terraform (HCL), логиката се унифицира. За целите на прототипната SaaS услуга, инфраструктурата трябва да обслужва публичен уеб трафик (HTTP на порт 80) и да позволява отдалечено административно управление чрез криптиран канал (SSH на порт 22).

1. Конфигурация в AWS (Основна среда): В екосистемата на Amazon Web Services, защитните правила се прилагат чрез "Групи за сигурност" (Security Groups). Те действат като запомнящи състоянието защитни стени на ниво виртуален мрежов интерфейс на самата инстанция.

```

1  # Дефиниране на Група за сигурност в AWS
2  resource "aws_security_group" "web_sg" {
3      name ..... = "fincloud-web-sg"
4      description = "Allow inbound HTTP and SSH traffic for FinCloud Primary"
5      ..
6      # Правило за входящ (Inbound) уеб трафик от крайните клиенти
7      ingress {
8          from_port... = 80
9          to_port..... = 80
10         protocol..... = "tcp"
11         cidr_blocks = ["0.0.0.0/0"] # Разрешава достъп от целия публичен интернет
12     }
13
14     # Правило за входящ (Inbound) административен достъп
15     ingress {
16         from_port... = 22
17         to_port..... = 22
18         protocol..... = "tcp"
19         cidr_blocks = ["0.0.0.0/0"] # В реална среда тук се задава само IP адресът на администратора
20     }
21
22     # Правило за изходящ (Outbound) трафик (напр. за изтегляне на системни актуализации)
23     egress {
24         from_port... = 0
25         to_port..... = 0
26         protocol..... = "-1" ..... # "-1" означава всички протоколи
27         cidr_blocks = ["0.0.0.0/0"]
28     }
29 }
30

```

2. Конфигурация в GCP (Резервна среда): В екосистемата на Google Cloud Platform, защитата се реализира чрез глобални правила на защитната стена (Firewall Rules), които се прикачват към самата VPC мрежа и филтрират трафика на базата на мрежови тагове (Network Tags), поставени върху виртуалните машини.

```

1  # Дефиниране на правило за защитна стена в GCP
2  resource "google_compute_firewall" "web_firewall" {
3      name = "fincloud-allow-http-ssh"
4      network = "default" # Насочва правилото към основната мрежа
5
6      # Разрешени протоколи и портове
7      allow {
8          protocol = "tcp"
9          ports = ["80", "22"]
10     }
11
12     # Източник на трафика (целият интернет)
13     source_ranges = ["0.0.0.0/0"]
14
15     # Правилото ще се приложи само върху машини с този етикет
16     target_tags = ["fincloud-web-node"]
17 }
18 |

```

Концептуална аналогия (Защитата на дигиталната сграда): Можем да разгледаме процеса по конфигуриране на облачните среди като строежа на строго охраняема корпоративна сграда (нашата SaaS платформа). Самото наемане на парцел за строеж е създаването на Виртуалната частна мрежа (VPC) - това поставя физическа ограда около нашия периметър. Групите за сигурност (Security Groups / Firewalls) представляват въоръжената охрана на входа на сградата. По подразбиране, охраната има нареждане: "Не пускай абсолютно никого". Чрез Terraform кода по-горе, ние даваме точни, писмени инструкции на охраната: "Пропускай през Главния вход (Порт 80) всички граждани, които искат да разгледат витрината на магазина, и пропускат през Служебния вход (Порт 22) само хората с административни пропуски". Всяка друга заявка за достъп (например опит за влизане през Порт 3306 за бази данни) бива автоматично блокирана и отхвърлена още на улицата, преди изобщо да достигне до вратата на нашия сървър.

5.3. Разгръщане на изчислителни ресурси

След като мрежовият фундамент и защитните периметри са успешно дефинирани, следващата фаза от реализацията обхваща разгръщането на самите изчислителни възли (Compute Nodes). В контекста на SaaS платформата FinCloud Analytics, това са виртуалните сървъри, които обработват бизнес логиката и обслужват крайните потребителски заявки.

Една от най-мощните концепции в подхода "Инфраструктура като код" (IaC) е възможността сървърите не просто да бъдат стартирани, но и напълно конфигурирани в момента на тяхното създаване, без необходимост системен администратор да влиза ръчно в тях. Този процес се нарича Bootstrapping. Чрез Terraform, към дефиницията на всяка виртуална машина се подава инициализационен скрипт (в AWS това се нарича `user_data`, а в GCP - `metadata_startup_script`). При първото зареждане на операционната система, този скрипт се изпълнява автоматично с най-високи системни права (`root`), като изтегля нужните софтуерни пакети, стартира уеб сървър и генерира визуалния интерфейс на SaaS приложението.

Кодова реализация (Terraform) на основния възел в AWS:

```
1 # Дефиниране на основната виртуална машина (EC2 Instance) в AWS
2 resource "aws_instance" "primary" {
3   ami           = "ami-0c02fb55956c7d316" # Идентификатор на Amazon Linux 2
4   instance_type = "t2.micro"
5
6   # Прикачване на създадената по-рано защитна стена
7   vpc_security_group_ids = [aws_security_group.web_sg.id]
8
9   # Инициализационен скрипт (Bootstrap), който се изпълнява при стартиране
10  user_data = <<-EOF
11  ...#!/bin/bash
12  ...sudo yum update -y
13  ...sudo yum install -y httpd
14  ...
15  ...sudo cat << 'HTML' > /var/www/html/index.html
16  ...<!DOCTYPE html><html><head><meta charset="UTF-8"><title>FinCloud Live</title>
17  ...<style>body{background-color:#0f172a; color:#f8fafc; text-align:center; padding-top:100px;}</style>
18  ...</head><body>
19  ...<h2 style="color:#10b981;" PRIMARY SYSTEM ACTIVE: AWS (Frankfurt)</h2>
20  ...<h1>Live Bitcoin (BTC) Price</h1>
21  ...<div style="font-size:60px; color:#10b981;" id="btc-price">$ 64,250.00</div>
22  ...<script>
23  ...  setInterval(() => {
24  ...    document.getElementById('btc-price').innerText = '$ ' + (64000 + Math.random() * 500).toFixed(2);
25  ...  }, 1500);
26  ...</script>
27  ...</body></html>
28  ...HTML
29
30  ...sudo systemctl start httpd
31  ...sudo systemctl enable httpd
32  ...EOF
33
34  tags = {
35    Name = "FinCloud-Primary"
36  }
37 }
```

5.4. Реализация на механизъм за автоматично пренасочване на трафика

Наличието на изчислителни ресурси в два независими облака (Amazon Web Services и Google Cloud Platform) само по себе си не осигурява непрекъсваемост на услугата. За да се реализира пълноценна устойчива архитектура, е необходим интелигентен механизъм, който да управлява потребителския трафик и да взема автономни решения накъде да го насочи в случай на криза. В монолитните традиционни системи пренасочването на трафика при авария често изисква ръчна промяна на DNS записите от страна на системен администратор - процес, който може да отнеме часове и води до значителни финансови загуби за SaaS платформата. За да се елиминира този риск, прототипът на FinCloud Analytics имплементира напълно автоматизиран механизъм за превключване при отказ (Failover Routing) на ниво Глобална система за имена на домейни (DNS).

Техническа имплементация чрез Infrastructure as Code:

Управлението на трафика се осъществява чрез услугата AWS Route 53. В Terraform конфигурацията са дефинирани два отделни DNS записа (тип "A" - IPv4 Address) за един и същи домейн (fin-cloud.click). Магията на маршрутизирането се крие в параметъра failover_routing_policy, който създава конфигурация тип Активен-Пасивен (Active-Passive):

```

1  # 1. ОСНОВЕН ЗАПИС (Primary Node) – Разположен в AWS
2  resource "aws_route53_record" "primary" {
3    zone_id = aws_route53_zone.main.zone_id
4    name    = "fin-cloud.click"
5    type    = "A"
6
7    # TTL (Time To Live) е настроен агресивно на 60 секунди за бързо опресняване
8    ttl     = "60"
9
10   # Динамично извличане на публичния IP адрес на AWS инстанцията
11   records = [aws_instance.primary.public_ip]
12
13   # Дефиниране на ролята като Основна инфраструктура
14   failover_routing_policy {
15     type = "PRIMARY"
16   }
17
18   set_identifier = "FinCloud-Primary-AWS"
19
20   # КРИТИЧНО: Този запис е валиден САМО ако здравната сонда връща статус "Healthy"
21   health_check_id = aws_route53_health_check.primary_check.id
22 }
23
24 # 2. РЕЗЕРВЕН ЗАПИС (Secondary Node) – Разположен в GCP
25 resource "aws_route53_record" "secondary" {
26   zone_id = aws_route53_zone.main.zone_id
27   name    = "fin-cloud.click"
28   type    = "A"
29   ttl     = "60"
30
31   # Динамично извличане на публичния IP адрес на машината в Google Cloud
32   records = [google_compute_instance.secondary.network_interface.0.access_config.0.nat_ip]
33
34   # Дефиниране на ролята като Резервна инфраструктура
35   failover_routing_policy {
36     type = "SECONDARY"
37   }
38
39   set_identifier = "FinCloud-Secondary-GCP"
40 }

```

5.5. Наблюдение, регистриране и известяване при проблеми

Механизмът за автоматично пренасочване, описан в предходната точка, е напълно "сляп", ако няма система, която да му казва кога е настъпила авария. За да се създаде истинска способност за самовъзстановяване, архитектурата се нуждае от "очи и уши". Тази роля се изпълнява от Здравните сонди (Health Checks). В контекста на облачните изчисления, мониторингът не трябва да се ограничава само до това дали сървърът е включен (Ping/ICMP на ниво мрежа). Сървърът може да работи, но уеб приложението (Nginx) да е забило. Затова

сондата на "FinCloud Analytics" е конфигурирана да извършва проверки на приложно ниво (Application Layer - Layer 7 по OSI модела).

Сондата изпраща реална HTTP заявка към главния сървър в AWS на всеки 30 секунди. Тя очаква сървърът да върне валиден HTTP статус код 200 OK (успех). За да се избегне превключване на трафика при моментно забавяне на интернет връзката (фалшива тревога), е въведен механизъм за праг на толерантност (Failure Threshold).

```
1 resource "aws_route53_health_check" "primary_check" {
2   .. # Сондата следи публичния IP адрес на основния AWS сървър
3   .. ip_address ..... = aws_instance.primary.public_ip
4   .. port ..... = 80
5   .. type ..... = "HTTP"
6   .. resource_path ..... = "/"
7   ..
8   .. # Времеви интервал: Прави проверка на всеки 30 секунди
9   .. request_interval .. = "30"
10  ..
11  .. # Праг на отказ: Сървърът трябва да не отговори 3 поредни пъти,
12  .. # преди да бъде обявен за "Unhealthy" (общо 90 секунди за детекция)
13  .. failure_threshold = "3"
14  ..
15  .. tags = {
16  ..   Name = "FinCloud-Primary-HealthCheck"
17  .. }
18 }
```

Когато прагът от 3 неуспешни опита бъде преминал, Route 53 незабавно маркира `aws_instance.primary` като деградиран ресурс. В този момент се задейства тригер (Trigger), който подава сигнал към Failover политиката от т. 5.4 да скъса връзката с AWS и да насочи потребителите към GCP. Чрез синергията между тези два компонента (Health Check + Failover Policy), прототипната инфраструктура постига ниво на автономност, характерно за високотехнологични Enterprise системи, елиминирайки напълно човешкия фактор в критичните първи минути на всяка мащабна авария.

5.6. Технически ограничения на прототипа

Всяка прототипна система (Proof of Concept - PoC), разработена за целите на академично изследване, се проектира с ясно дефинирани граници. Целта е да се изолира, тества и докаже основната научна хипотеза - в нашия случай ефективността на автоматизирания многооблачен Failover без да се генерира излишна архитектурна сложност или неоправдани финансови разходи. Във връзка с това, при реализацията на референтната архитектура за FinCloud Analytics са допуснати три съзнателни технически ограничения, които трябва да бъдат отчетени при евентуално мащабиране на системата до реална производствена (Enterprise) среда:

1. Ефимерно управление на мрежовите адреси (Dynamic vs. Static IPs)

В текущия код виртуалните машини в Amazon Web Services (AWS) и Google Cloud Platform (GCP) използват динамични публични IP адреси. Това означава, че при рестартиране на инстанцията (напр. след спиране от конзолата), облачният доставчик освобождава стария адрес и присвоява нов.

Корпоративното решение: В реална бизнес среда този проблем се решава чрез резервиране на статични адреси - Elastic IPs (в AWS) и Static External IPs (в GCP). Тъй като тези статични ресурси се таксуват постоянно (дори когато машината е спряна), те са изключени от прототипа с цел минимизиране на оперативните разходи (OPEX). Terraform кодът е написан така, че динамично да извлича новото IP и да го подава към Route 53, заобикаляйки това ограничение.

2. Липса на транспортно криптиране (SSL/TLS Termination)

Комуникацията между крайните потребители и уеб сървърите (Nginx/Apache) се осъществява по некриптиран протокол (HTTP през порт 80). Липсва инсталиран SSL сертификат за HTTPS.

Отражение върху системата: Поради строгите политики за сигурност на съвременните уеб браузъри (HSTS), понякога заявките към домейна fin-cloud.click могат да бъдат форсирани към порт 443 (HTTPS), което води до локален таймаут (Connection Timed Out). В реална SaaS платформа, криптирането би се управлявало централизирано от Application Load Balancer или Cloudflare, преди трафикът да достигне до самите виртуални машини.

3. Абстракция на слоя за съхранение на данни (Stateless Architecture)

Прототипът успешно демонстрира устойчивостта на мрежовия и изчислителния слой. Уеб възлите (Compute nodes) са проектирани като безсъстоятелни те визуализират интерфейса, но не съхраняват транзакционни клиентски данни локално.

Перспектива за развитие: За да функционира една финансова система пълноценно, тя изисква слой за съхранение (Database Layer). В многооблачна архитектура това би наложило внедряването на глобално разпределена база данни (например CockroachDB или репликация между AWS RDS и Google Cloud SQL), която да синхронизира информацията в реално време между двата облака. Тази конфигурация е извън обхвата на настоящата теза.

ГЛАВА 6. ЕКСПЕРИМЕНТАЛНО ИЗСЛЕДВАНЕ И ОЦЕНКА

6.1. Методика на тестване

За да се докаже по безспорен начин, че проектираната многооблачна инфраструктура изпълнява заложените нефункционални изисквания (особено Целевото време за възстановяване - RTO), е възприет методът на контролираната симулация на отказ (Disaster Injection). Този подход, популяризиран в индустрията като "Хаос инженерство" (Chaos Engineering), се състои в умишлено предизвикване на тежка повреда в производствената среда, за да се наблюдава и измери реакцията на автоматизираните защитни механизми.

Методиката на тестване преминава през три строго дефинирани етапа:

Потвърждаване на базовото състояние (Baseline Verification): Верификация, че системата обслужва трафика нормално през основния IaaS доставчик.

Индукциране на критичен отказ (Fault Injection): Симулиране на прекъсване на AWS центъра за данни.

Измерване на метриките (Telemetry & RTO Assessment):

Хронометриране на времето от момента на срива до момента на успешното зареждане на платформата от резервния GCP център.

6.2. Работа при нормален режим

Експериментът стартира с анализ на системата в нейния нормален експлоатационен статус. При тези условия, цялата архитектура функционира оптимално и маршрутизираният слой изпълнява ролята си без аномалии. Визуален и приложен слой: Потребителят получава незабавен достъп до презентационния слой на FinCloud Analytics. Интерфейсът е оцветен в зелено с ясен маркер "PRIMARY SYSTEM ACTIVE: AWS (Frankfurt)". Този маркер доказва, че HTTP заявката е успешно получена, обработена и върната от EC2 инстанцията в инфраструктурата на Amazon. Алгоритъмът за извличане на Биткойн цените работи без закъснение.

Контролен и мониторингов слой: В контролния панел на глобалния маршрутизатор (AWS Route 53), Здравната сонда (Health Check), прикрепена към AWS сървър, показва статус Healthy (Здрав). Графиката за наличност (Health status) е стабилна на ниво.

Това базово състояние потвърждава, че DNS Failover политиката държи резервния маршрут към Google Cloud Platform (GCP) строго "заклучен" и в режим на изчакване (Пасивен), предотвратявайки разпиляване на трафика. Системата е напълно подготвена за предстоящия стрес-тест.

6.3. Симулация на отказ на доставчик

За да се докаже на практика, че архитектурата притежава заложената способност за самовъзстановяване, се преминава към същинската фаза на експеримента - симулиране на катастрофална авария в основния център за данни AWS.

Механика на симулацията:

Действие: Администраторът влиза в контролния панел на Amazon EC2 и подава команда за принудително спиране (Stop Instance) на основната виртуална машина FinCloud-Primary.

Технически ефект (Какво се случва): За няколко секунди, операционната система на виртуалната машина спира процесите си. Уеб сървърът (Nginx/Apache), който до този момент е слушал активно на порт 80, е "убит". Публичният IP адрес на машината спира да отговаря на мрежови заявки.

Потребителско изживяване (Деградация):

Ако в този конкретен момент краен клиент се опита да отвори или презареди (<http://fin->

cloud.click), неговият браузър ще направи опит да се свърже със зеления (AWS) сървър. Тъй като сървърът вече не съществува в мрежовото пространство, заявката увисва и след определен период браузърът връща грешка ERR_CONNECTION_TIMED_OUT. На този етап услугата е официално прекъсната.

Ние умишлено натискаме бутона на алармата (Спираме EC2 инстанцията). В този момент нормалният работен процес е прекъснат (получаваме Timeout). Целта на експеримента е да докажем, че веднага след натискането на алармата, автоматичните противопожарни врати се отварят сами, асансьорите слизат на партера и хората биват безопасно евакуирани (трафикът се пренасочва) без нито един мениджър да трябва да дава команди по мегафона.

6.4. Измерване на времето за възстановяване

След като сригът е успешно инициран, най-критичната част от експеримента е количественото измерване на Целевото време за възстановяване (Recovery Time Objective - RTO). RTO дефинира максимално допустимото време, през което SaaS платформата може да бъде недостъпна, преди да започне да генерира финансови загуби или да наруши Service Level Agreement (SLA) договорите с клиентите. Тук трябва да направим забележка, че при продуктивна среда, в която се плаща за по-добри и по-мощни услуги от облачните доставчици - времето драстично пада в полза на компанията. Забавянето при превключването в многооблачната архитектура не е случайно или хаотично - то е функция на строги математически параметри, предварително дефинирани в Terraform кода.

00 секунди: AWS инстанцията е спряна. Сайтът е недостъпен.

10 до 30 секунди (Фаза на изчакване): Здравната сонда (Health Check) в Route 53 изпраща поредния си HTTP Ping. Тя не получава отговор. Тъй като в кода сме задали `failure_threshold = 3` и `request_interval = 30`, сондата не вдига тревога веднага. Тя "помисля", че това може да е моментно трептене на интернета.

90 секунди (Детекция на срига): Сондата отчита третия пореден неуспешен опит ($3 \times 30 \text{ сек} = 90 \text{ секунди}$). Метриката за проверяване на статуса (Health status) рязко пада на 0%. Route 53 маркира AWS като Нездравословен (Unhealthy). Автоматичният Failover се задейства и маршрутизаторът спира да раздава AWS адреса, започвайки да промотира резервния GCP адрес.

90 до 150 секунди (DNS Пропагация): Тук се намесва стойността TTL (Time to Live) от 60 секунди, която сме задали на DNS записа. Дори Route 53 да е превключил трафика вътрешно,

компютрите на клиентите по света (и техните интернет доставчици) пазят "кеш" със стария, съхранен адрес. Тези кешове изтичат средно за около 60 секунди.

Обобщен резултат:

Реално отчетеният RTO по време на експеримента е между 2.5 и 3 минути (90 секунди за детекция + ~60 секунди за разпространение на новия запис).

Този резултат е изключително успешен за проект без огромни корпоративни бюджети. За сравнение, ръчното възстановяване на услуга (включващо диагностика на проблема от екип по поддръжката, влизане в конзолата, пренасочване на IP адреси и рестартиране) обикновено отнема между 30 и 120 минути в стандартни монолитни среди. С постигнатите 3 минути RTO, архитектурата на FinCloud Analytics доказва способността си да защити критичните финансови процеси на своите клиенти от продължителни прекъсвания, като запазва висока степен на наличност и конкурентно предимство.

6.5. Сравнителен анализ на резултатите

За да се оцени обективно приносът на предложената многооблачна архитектура, резултатите от експеримента (постигнатото Целево време за възстановяване от под 3 минути) трябва да бъдат съпоставени с алтернативните традиционни подходи за управление на кризи в монолитни среди.

Сравнителният анализ разкрива три фундаментални предимства на реализирания прототип:

1. Елиминиране на човешкия фактор (Zero-Touch Recovery)

При традиционната монолитна архитектура (Single-Cloud), възстановяването при локален срив често изисква ръчна интервенция. Системният администратор трябва да получи известие (аларма), да анализира лог файловете, да локализира проблема и ръчно да пренасочи трафика или да рестартира услуги. Този процес, известен като Mean Time To Repair (MTTR), често отнема часове. При референтната архитектура на FinCloud Analytics, човешкият фактор е напълно изключен в критичната фаза. Механизмът на AWS Route 53 взема детерминирано решение на базата на Health Check алгоритъма и изпълнява превключването автономно.

2. Редундантното на ниво "Доставчик"

Дори най-добре проектираните архитектури в рамките на един доставчик (например използване на множество Availability Zones в AWS) са уязвими към т.нар. глобални сринове в контролния панел на самия доставчик (Control Plane Failures). Експериментът доказва, че чрез използването на два независими корпоративни субекта (Amazon и Google), архитектурата постига истинска устойчивост и елиминира риска от технологична зависимост (Vendor Lock-in).

3. Успешна синхронизация на състоянието (Процесът Failback)

Освен Failover (превключване при криза), експериментът валидира и процеса на Failback (завръщане към нормален режим). Когато AWS инстанцията бъде стартирана отново и мрежовите услуги са възстановени, здравната сонда отново засича статус 200 OK. Тъй като AWS записът е с политика PRIMARY, Route 53 автоматично деприоритизира GCP и връща целия потребителски трафик обратно към основната инфраструктура. Този двупосочен автоматизъм е върхът в изграждането на High Availability системи.

6.6. Оценка на разходите за поддръжка

Всяко архитектурно решение в корпоративния свят трябва да бъде не само технически издържано, но и икономически обосновано. Преходът от капиталови разходи (CAPEX) към гъвкави оперативни разходи (OPEX) е едно от главните предимства на облачните изчисления, но неправилното управление на многооблачни среди може да доведе до драстично превишаване на бюджетите. В рамките на настоящата разработка е приложена концепцията FinOps (Financial Operations) - стратегическо управление и оптимизация на облачните разходи. Оценката на разходите за прототипната система се базира на избрания модел "Топъл резерв" (Warm Standby).

Икономически анализ на Warm Standby срещу Active-Active

Модел Active-Active (Най-скъпият подход): При него и двата облака работят на 100% мощност едновременно, като трафикът се разпределя 50/50. Това изисква поддържането на скъпи, високопроизводителни сървъри както в AWS, така и в GCP, както и плащане за глобални балансатори на натоварването (Load Balancers). Този модел ефективно удвоява месечните разходи (OPEX) на организацията.

Нашият модел (Warm Standby): Резервната инфраструктура в GCP е проектирана да бъде изключително "лека" откъм ресурси. Използвана е виртуална машина от най-нисък клас (e2-micro), която консумира минимално количество CPU и RAM, докато е в пасивен режим.

Финансовите ползи за организацията:

Изградената архитектура доказва, че особено за малки и средни предприятия (МСП), не е необходимо да се харчат колосални суми за постигане на корпоративно ниво на сигурност.

Разходите за поддръжка на тази система се свеждат до:

Символична месечна такса за поддържане на Hosted Zone и Health Checks в AWS Route 53 (обикновено под \$1 на месец).

Заплащане за базовото време на работа (Uptime) на e2-micro инстанцията в Google Cloud, което често попада дори в безплатния лимит (Free Tier) на доставчика.

Използването на Terraform е напълно безплатно (Open Source версията), което елиминира разходите за допълнителен софтуер за автоматизация.

ЗАКЛЮЧЕНИЕ

Настоящата магистърска теза разглежда, анализира и експериментално доказва необходимостта от фундаментална промяна в парадигмата за проектиране, изграждане и управление на съвременни "Софтуер като услуга" (SaaS) архитектури. В епохата на ускорена дигитална трансформация, непрекъсваемостта на бизнес процесите вече не е просто пожелателна техническа характеристика, а абсолютно задължително условие за корпоративно оцеляване, конкурентоспособност и запазване на доверието на крайните потребители.

Синтез на теоретичните изследвания и дефиниране на глобалния проблем

Проведеното в първите глави теоретично изследване категорично очерта еволюционния път на информационните инфраструктури - от тремавите, локално базирани монолитни системи (On-Premise), изискващи огромни капиталови инвестиции (CAPEX), до ерата на публичните облачни изчисления (Cloud Computing). Този преход безспорно демократизира достъпа до неограничена изчислителна мощност и прехвърли тежестта към по-гъвкавия модел на оперативните разходи (OPEX). Въпреки тези безспорни предимства, теоретичният анализ дефинира един критичен, често подценяван риск за съвременните организации: Илюзията за абсолютна надеждност на единичния облачен доставчик.

Както беше разгледано в разработката, изграждането на критична финансова или бизнес система (каквото е референтната платформа „FinCloud Analytics“) ексклузивно върху инфраструктурата на един единствен технологичен гигант (Single-Cloud стратегия) създава две фундаментални уязвимости:

1. Единична точка на мащабен отказ (Global Single Point of Failure - SPOF): Дори при наличие на локална резервираност (множество Availability Zones) в рамките на един доставчик, мащабни срывове в контролните панели (Control Planes) или компрометиране на вътрешните мрежи на самия доставчик могат да доведат до глобално прекъсване на услугата.
2. Зависимост от доставчика (Vendor Lock-in): Използването на проприетарни (затворени) технологии и API интерфейси лишава организацията от гъвкавост, превръщайки я в "заложник" на ценовите и технологични политики на съответния доставчик.

Концептуално обобщение (Аналогията за дигиталната империя): Монолитната, еднооблачна архитектура може да бъде оприличена на изграждането на великолепна, модерна столица, която обаче е напълно зависима от един единствен акведукт за доставка на вода. Колкото и съвършена да е вътрешната водопроводна мрежа на града, ако източникът (доставчикът) пресъхне или бъде разрушен, целият град спира да функционира. Чрез настоящата магистърска теза ние доказахме, че съвременната дигитална "империя" трябва да черпи ресурси от множество независими източници (Multi-Cloud), за да гарантира своето безпроблемно съществуване.

Архитектурната алтернатива: Многооблачен и Поли-облачен подход. В отговор на дефинирания проблем, изследването защита тезата, че бъдещето на критичните SaaS решения лежи в имплементацията на Многооблачни (Multi-Cloud) и Поли-облачни (Poly-Cloud) архитектури. Чрез абстрахиране на приложната логика от базовия инфраструктурен слой - използвайки стандартизирани технологии като контейнеризация и инструменти за оркестрация като Terraform - организациите възвръщат своя дигитален суверенитет. Те придобиват

способността да разпределят риска между независими корпоративни субекти (в нашия случай Amazon Web Services и Google Cloud Platform), елиминирайки напълно опасността от vendor lock-in.

Практическа реализация и доказателство на концепцията (Proof of Concept)

Теоретичните постановки, дефинирани в първата част на изследването, бяха успешно доказани в реално функционираща прототипна архитектура (Proof of Concept). Разработената система, симулираща финансовата SaaS платформа „FinCloud Analytics“, демонстрира на практика как сложните концепции за разпределени изчислителни среди могат да бъдат внедрени с висока степен на надеждност и предвидимост.

Ключов фактор за успеха на тази реализация беше възприемането на парадигмата "Инфраструктура като код" (Infrastructure as Code - IaC). Чрез използването на HCL (HashiCorp Configuration Language) и инструмента Terraform, процесът по изграждане на сложни мрежови топологии (Virtual Private Clouds), защитни стени (Security Groups) и виртуални изчислителни възли беше напълно абстрахиран от ръчни процеси и превърнат в детерминиран, проследим и повторяем софтуерен артефакт. Този подход доказва, че човешкият фактор - основен източник на конфигурационни грешки в ИТ индустрията - може да бъде елиминиран при разгръщането на критични архитектури.

Оценка на експерименталните резултати и постигане на устойчивост и RTO

Кулминацията на настоящата магистърска теза експерименталното изследване чрез симулация на катастрофален отказ (Disaster Injection). Целта на този тест бе да валидира най-важната нефункционална характеристика на системата - способността ѝ за автоматизирано самовъзстановяване (Self-healing) и превключване при отказ (Failover). Регистрираният инцидент задейства автоматизираната политика за маршрутизиране, която мигновено и без човешка намеса прекъсна насочването на потребителския трафик към компрометирания AWS възел и го пренасочи към резервната инфраструктура (Google Cloud Platform) и постигаме минимизиране на целевото време за възстановяване (RTO): Най-значимият метричен резултат от експеримента е постигнатото RTO от под 3 минути. В контекста на финансовите SaaS услуги, където всяка минута прекъсване води до съществени финансови и репутационни щети, този резултат доказва, че дори малки и средни организации могат да постигнат ниво на

непрекъсваемост (High Availability), съизмеримо с това на големите корпорации.

Доказана сигурност и мрежова изолация

Освен устойчивостта, реализираният прототип валидира и концепцията за "Нулево доверие" (Zero Trust) в многооблачна среда. Дефинираните чрез код правила за защитна стена (Firewall Rules) както в AWS, така и в GCP, гарантираха строга мрежова изолация. Достъпът до изчислителните възли бе лимитиран единствено до абсолютно необходимите протоколи (HTTP и SSH), демонстрирайки, че повишената архитектурна сложност на Multi-Cloud подходите не компрометира сигурността, ако тя е заложена декларативно (Secure by Design).

Икономическа ефективност и оптимизация на разходите (Cloud FinOps)

Изследването категорично затвърди икономическите предимства на облачните технологии, свързани с фундаменталната трансформация на ИТ бюджетирането от фиксирани капиталови инвестиции (CAPEX) към динамични и гъвкави оперативни разходи (OPEX). Въпреки това, теоретичният анализ посочи, че безконтролното мащабиране в мулти-облачни среди може лесно да доведе до преразход и финансова неефективност. За да се адресира този проблем, прототипната реализация бе проектирана през призмата на съвременните FinOps (Financial Operations) практики. Изборът на архитектурен модел "Топъл резерв" (Warm Standby), вместо традиционния "Активен-Активен" (Active-Active) модел, демонстрира интелигентен компромис между критичната нужда от непрекъсваемост и финансовите ограничения на организацията. Поддържането на минимално оразмерени изчислителни ресурси (като e2-micro инстанциите) в резервния облак, които се мащабират автоматично само при реално настъпване на аварийно събитие, доказва, че малките и средни предприятия (МСП) могат да си позволят корпоративно (Enterprise) ниво на сигурност и резервираност. Разходите за превенция на срива са фрагмент от потенциалните финансови загуби, които организацията би претърпяла при спиране на своята SaaS услуга дори за няколко часа.

Нормативно съответствие (Compliance) и регулаторна устойчивост (DORA & GDPR)

В контекста на финансово-технологичния (FinTech) сектор, към който принадлежи референтната организация „FinCloud Analytics“, техническата устойчивост е неразривно

свързана със стриктното нормативно съответствие. Настоящата магистърска теза разгледа и доказва приложимостта на предложената архитектура спрямо най-новите и строги регулаторни рамки на Европейския съюз. Разработената многооблачна инфраструктура адресира директно ключовите изисквания на АКТА за цифрова оперативна устойчивост на финансовия сектор (Digital Operational Resilience Act - DORA). DORA изисква от финансовите институции не просто да имат резервни планове, но активно да управляват рисковете, свързани с външни доставчици на ИКТ услуги (включително облачни платформи), и да притежават ясни "стратегии за изход" (Exit Strategies). Чрез абстрахирането на конфигурацията чрез Terraform и доказаната способност за прехвърляне на операциите от AWS към GCP в рамките на минути, организацията изпълнява това регулаторно изискване, елиминирайки риска от системна концентрация при един доставчик. Паралелно с това, архитектурата подкрепя изискванията на Общия регламент относно защитата на данните (GDPR), като предоставя на организацията пълен "дигитален суверенитет" - възможността да контролира прецизно в кои географски региони (напр. Франкфурт и Копенхаген) се разполагат изчислителните мощности, гарантирайки, че данните не напускат юрисдикцията на ЕС дори при сценарий на възстановяване след авария (Disaster Recovery).

Бизнес стойност и стратегическа гъвкавост

В обобщение на бизнес анализа, стойността на разработената многооблачна архитектура надхвърля чисто техническите параметри (като постигнатото RTO). Тя предоставя на организацията стратегическа гъвкавост. Предотвратяването на технологичната зависимост (Vendor Lock-in) означава, че компанията запазва своята преговорна сила спрямо технологичните гиганти. Ако основният доставчик (AWS) промени драстично ценовата си политика или деградира качеството на услугите си, "FinCloud Analytics" разполага с напълно функционална, паралелна инфраструктура, към която може да мигрира трайно и безпроблемно. Това трансформира ИТ инфраструктурата от поддържащ разходен център (Cost Center) в ключов бизнес актив (Business Asset), който генерира доверие у клиентите и конкурентно пазарно предимство.

Концептуално обобщение (Аналогията за юридическия договор):

Регулаторната устойчивост в монолитния облак е като да подпишете доживотен договор с един единствен доставчик на електричество, без клауза за прекратяване. Ако доставчикът вдигне цените тройно или мрежата му започне да аварира ежедневно, вашият бизнес е застрашен, но законът не ви позволява да го напуснете. Изградената архитектура с отворени инструменти (Terraform) и многооблачно маршрутизиране (Route 53) играе ролята на перфектно съставен юридически договор с "вратичка". Вие ползвате услугите на доставчика, но във всяка една секунда държите ключа (кода) в ръцете си и сте готови да преместите целия си бизнес на друго място, отговаряйки безупречно на изискванията на одиторите и регулаторите (DORA/GDPR).

Насоки за бъдещо развитие

Разработената в рамките на тази магистърска теза архитектура полага солидна, но базова основа. Нейното надграждане до пълноценно корпоративно (Enterprise-grade) решение открива множество възможности за бъдещи изследвания и инженерни подобрения:

1. Интеграция на Глобално разпределени бази данни: Еволюцията на архитектурата изисква внедряването на гео-репликирани бази данни (напр. CockroachDB, Amazon Aurora Global Database или репликация между AWS RDS и Google Cloud SQL). Това би осигурило консистентност на данните (Data Consistency) в реално време между двата облака, гарантирайки, че при Failover клиентите няма да загубят своите последни финансови транзакции.

2. Миграция към Контейнерна оркестрация (Kubernetes): За повишаване на преносимостта (Portability) и улесняване на процесите по непрекъсната интеграция и доставка (CI/CD), виртуалните машини (EC2/Compute Engine) могат да бъдат заменени с управлявани Kubernetes клъстери (Amazon EKS и Google GKE). Това би позволило изпълнението на същинска микросървисна архитектура, където отделните компоненти на "FinCloud Analytics" се мащабират независимо един от друг.

3. Внедряване на "Infrastructure as Code" пайплайни (GitOps): Локалното изпълнение на Terraform кода може да бъде автоматизирано чрез платформи като GitHub Actions или GitLab CI/CD. Този подход (GitOps) би позволил промените в инфраструктурата да се тестват, валидират и внедряват автоматично при всяко качване на нов код в хранилището (Repository).

Финално обобщение и научен принос

В заключение, настоящата магистърска теза постигна своята основна цел: да анализира, проектира и докаже ефективността на многооблачните (Multi-Cloud) и поли-облачните (Poly-Cloud) архитектури като инструмент за постигане на висока наличност и устойчивост в съвременните SaaS екосистеми. Научно-приложният принос на разработката се състои в успешното обединяване на теоретичните концепции за разпределени системи с практическите методи на инфраструктурата като код. Изградената архитектура с модел "Топъл резерв" доказва емпирично, че е възможно да се постигне целево време за възстановяване от под 3 минути, елиминирайки единичните точки на отказ (и риска от зависимост от един доставчик. Чрез синергията между глобалното DNS маршрутизиране (AWS Route 53), автономните здравни сонди (Health Checks) и инфраструктурната автоматизация (Terraform), изследването предоставя на съвременните организации ясен, икономически обоснован (FinOps оптимизиран) и регулаторно издържан (DORA съвместим) модел за изграждане на устойчиви дигитални услуги. Този модел не просто предпазва бизнеса от технологични катастрофи, но му осигурява гъвкавостта, необходима за иновации и лидерство в конкурентния ИТ сектор.

ЛИТЕРАТУРА

- [1] Gartner Research. (2023). Forecast: Public Cloud Services, *Worldwide, 2021-2025*. Gartner Inc.
- [2] Opara-Martins, J., Sahandi, R., & Tian, F. (2016). *Critical analysis of vendor lock-in and its impact on cloud computing migration*. Journal of Cloud Computing.
- [3] Mell, P., & Grance, T. (2011). *The NIST Definition of Cloud Computing*. National Institute of Standards and Technology (NIST).
- [4] Buyya, R., et al. (2009). *Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility*. Future Generation Computer Systems.
- [5] Artac, M., et al. (2017). *DevOps: introducing infrastructure-as-code*. IEEE/ACM International Conference on Software Engineering Companion.
- [6] FinOps Foundation. (2021). *What is FinOps? Cloud Financial Management*. The Linux Foundation.
- [7] HashiCorp. (2023). *Infrastructure as Code in a Multi-Cloud Environment*. HashiCorp Whitepapers.